Embedded Systems Project Planning

https://gemini.google.com/app/f28bf499147f03bb

User prompt: I am working on a new project for my embedded class. the project description and requirements are in the pdf file i will be attaching to this message.

Response: The ECSE4235_PARMCO_Project.pdf file outlines the PARMCO (Phone App RP4 Motor Control) project, a closed-loop stepper motor system.

- The system architecture uses a Raspberry Pi 4 to drive a Nema17 stepper motor via an SN754410NE H-bridge, isolated by a 4N25 optocoupler.
- Position and speed feedback for the control loop rely on an AS5600 magnetic encoder and an IR sensor reading a rotating propeller.
- A custom phone app communicates with the RP4 over Bluetooth to issue commands, set modes (Manual or Maintain), and display status data like the IR RPM and encoder angle.
- A strict ground rule is that AI must act as the primary author for all software, hardware schematics, and documentation. Human team members manage the AI and are only allowed to make clearly marked, minimal tweaks.

The software stack requires the time-critical step, enable, and step-timing loop routines to be written in ARM assembly using standard AAPCS calling conventions. These assembly routines are then called from a C application handling the Bluetooth communication, sensor reading, and control math. This architecture naturally extends the low-level ARM assembly, memory-mapped GPIO access, and C driver development you already build on the Raspberry Pi 4.

Checkpoint 1 is scheduled for Tuesday, October 20. For this milestone, the hardware must successfully turn the motor using required keyboard commands, and you need to complete a stall-speed characterization alongside the strategy and pre-power inspection documents.

Do you want to start by generating the SN754410NE motor circuit schematic and parts list, or should we begin outlining the ARM assembly and C structure for the Checkpoint 1 keyboard commands?

User prompt: the team is composed of me and one other lab partnet. currenlty, my lab partner is starting on the hardware side of checkpoint one. We will be focusing on the software section for the first checkpoint, for the time being. To start, narrow down the scope of the current tasks to checkpoint one

Response: For Checkpoint 1, the software scope centers on building a standalone C program with ARM assembly routines to control the motor via specific keyboard commands.

- **C Program Structure and Keyboard Interface:** The C code must parse terminal keystrokes to trigger the required motor commands: enable/disable, direction (CW/CCW), one revolution, run/stop, and faster/slower (adjusting the step rate by 5% or less per press). The program also needs a status display that prints the chosen step mode (full or half) at startup and outputs the current step rate in steps per second whenever the speed changes.
- **ARM Assembly Motor Drivers:** The interface between the RP4 and the SN754410NE H-bridge must be written in assembly. This includes the step routine (looking up the next coil pattern based on direction and step mode), the enable routine (driving the 4N25 optocoupler), and the step-timing loop utilizing the ARM generic timer. Unlike direct memory-mapped GPIO access, these assembly routines must call the class RP4 library's GPIO functions using standard AAPCS conventions.
- **Strategy Document (Human-Written):** You need to submit a PDF detailing the phone platform and development tools for the app, your Bluetooth connection strategy, and the path to Checkpoint 2. It must also cover your team's work division, schedule, and how you plan to document the AI interactions.

You can set up the project structure using Makefiles and Git over your VS Code SSH session to link the C and assembly files. Which part of the software would you like to build first?

User prompt: since all requirements are known and well defined, we will start from the ground up, with higharchy similar to FPGA logic design. The top level file, written in C, will primarily handle IO and call lower level arm program functions to handle motor control and signal feedback

Response: Structuring the PARMCO software with a top-level C controller and lower-level ARM assembly drivers cleanly separates the application logic from the hardware interface. This modular approach aligns with the Checkpoint 1 requirements for standard AAPCS calling conventions between C and assembly.

System Hierarchy

- **`main.c` (Top-Level Controller):** Acts as the primary state machine. It implements a non-blocking terminal polling loop to capture the required keys for enable/disable, direction, one revolution, run/stop, and faster/slower. It maintains global variables for step rate, direction, and step mode (full or half), and triggers the status display outputs upon state changes.
- **`motor_driver.s` (Hardware Abstraction):** Contains the time-critical ARM assembly routines. It exposes global function labels to C:
  - `enable_routine`: Takes a binary argument from C to turn the H-bridge on or off through the 4N25 optocoupler.
  - `step_routine`: Takes direction and step mode, indexes into a predefined coil pattern table, and drives the SN754410NE inputs. It must branch to the class RP4 library's GPIO functions using standard ARM calling conventions rather than direct memory-mapped access.

- `step_timing_loop`: Reads the ARM generic timer to hold execution until the next step is due, based on the step rate passed down from the C controller.

To make the top-level C controller responsive to Checkpoint 1 keyboard commands while the motor is in continuous run mode, `main.c` will need to configure the Linux terminal into raw, non-blocking mode (typically via `termios`). This ensures keystrokes like "faster" or "stop" are registered immediately without halting the motor timing.

Should we draft the C state machine and non-blocking terminal I/O first, or start with the ARM assembly coil pattern lookup table and AAPCS step routine?

User prompt: lets start with the motor driver so that we can begin characterizing the motor stall as well with a test c program afterwards. the programs will be run on a raspberry pi 4 on ARM8 legacy 32 bit os. For each function, they need to be generalized to allow reuse and easy edits. Such generalization includes passing which pin to control as a parameter

Response: **`motor_driver.s` Architecture**

To support the 32-bit legacy ARM OS and generalize the driver for reusable pin assignments, the assembly routines accept GPIO pins and states as arguments in registers `r0`-`r3` per the AAPCS standard. This structure provides the step routine, enable routine, and step-timing loop required by the project specifications while relying on the class RP4 library for the actual GPIO register writes.

The step routine utilizes an internal state variable to track the coil sequence and looks up the next coil pattern in a predefined data table based on the direction and step mode.

```
.data
.align 2

@ Tracks the current position in the step sequence (0-7)
current_step:
    .word 0

@ Full-step coil patterns (4 steps: AB, BC, CD, DA)
@ Bits correspond to coils: 3=A, 2=B, 1=C, 0=D
full_step_table:
    .byte 0b1010, 0b0110, 0b0101, 0b1001

@ Half-step coil patterns (8 steps)
half_step_table:
    .byte 0b1000, 0b1010, 0b0010, 0b0110, 0b0100, 0b0101, 0b0001, 0b1001

.text
.global enable_routine
.global step_routine
.global step_timing_loop

@ ------------------------------------------------------------------------
@ enable_routine
@ Turns the H-bridge on or off through the 4N25 optocoupler[cite: 8, 9].
@ Args:
@   r0: Enable pin number
@   r1: State (1 = Enable, 0 = Disable)
@ ------------------------------------------------------------------------
enable_routine:
    push {lr}
    @ AAPCS: r0 and r1 already contain pin and state for the class library
    bl class_gpio_write    @ Replace with exact class library function name
    pop {pc}

@ ------------------------------------------------------------------------
@ step_routine
@ Looks up the next coil pattern and drives the SN754410NE inputs[cite: 9].
@ Args:
@   r0: Direction (1 = CW, 0 = CCW)
@   r1: Mode (1 = Half step, 0 = Full step)
@   r2: Pointer to integer array containing the 4 motor pins {A, B, C, D}
@ ------------------------------------------------------------------------
step_routine:
    push {r4-r8, lr}

    mov r4, r0             @ Save direction
    mov r5, r1             @ Save mode
    mov r6, r2             @ Save pin array pointer pointer

    @ Load current step index
```

```
        ldr r7, =current_step
        ldr r8, [r7]

        @ Update step index based on direction
        cmp r4, #1
        addeq r8, r8, #1       @ If CW, increment
        subne r8, r8, #1       @ If CCW, decrement

        @ Handle wrapping based on mode
        cmp r5, #1
        beq half_step_wrap

    full_step_wrap:
        and r8, r8, #3         @ Wrap 0-3 for full step (modulo 4)
        ldr r4, =full_step_table
        b fetch_pattern

    half_step_wrap:
        and r8, r8, #7         @ Wrap 0-7 for half step (modulo 8)
        ldr r4, =half_step_table

    fetch_pattern:
        @ Store updated step index
        str r8, [r7]

        @ Fetch the current bit pattern from the table
        ldrb r4, [r4, r8]      @ r4 now holds the 4-bit pattern

        @ Drive Pin A (Bit 3)
        ldr r0, [r6, #0]       @ Load Pin A number from array
        and r1, r4, #0b1000
        lsr r1, r1, #3
        bl class_gpio_write

        @ Drive Pin B (Bit 2)
        ldr r0, [r6, #4]       @ Load Pin B number
        and r1, r4, #0b0100
        lsr r1, r1, #2
        bl class_gpio_write

        @ Drive Pin C (Bit 1)
        ldr r0, [r6, #8]       @ Load Pin C number
        and r1, r4, #0b0010
        lsr r1, r1, #1
        bl class_gpio_write

        @ Drive Pin D (Bit 0)
        ldr r0, [r6, #12]      @ Load Pin D number
        and r1, r4, #0b0001
        bl class_gpio_write

        pop {r4-r8, pc}

    @ ---------------------------------------------------------------------------
    @ step_timing_loop
    @ Waits until the next step is due using the ARM generic timer[cite: 9].
    @ Args:
    @   r0: Target wait interval in timer ticks
    @ ---------------------------------------------------------------------------
    step_timing_loop:
        push {r4, lr}
        mov r4, r0             @ Save target interval

        @ Read current 32-bit timer value (CNTPCT lower half) into r1
        mrc p15, 0, r1, c14, c0, 0
        add r4, r1, r4         @ Calculate target end time

    wait_loop:
        mrc p15, 0, r1, c14, c0, 0
        cmp r1, r4
        blt wait_loop          @ Keep looping until current time >= end time

        pop {r4, pc}
```

Implementation Notes for the C Controller

- **Pin Array Wrapping:** The `step_routine` expects the four SN754410NE control pins to be passed as an array pointer. In your C test program, declare this as `int motor_pins[4] = {PIN_A, PIN_B, PIN_C, PIN_D};` and pass `motor_pins` as the third argument.
- **Timer Ticks Calculation:** The `step_timing_loop` expects raw timer ticks, not milliseconds or microseconds. The C test program must calculate the correct tick delay by dividing the ARM generic timer frequency (typically 54 MHz on the RPi4) by your target steps per second to achieve the desired step rate.
- **Library Integration:** Ensure you change `class_gpio_write` in the assembly file to match the exact function signature provided in the class RP4 library.

User prompt: attached is the class library documenation, for future reference

Response: This documentation perfectly bridges the gap between the C controller and the assembly drivers.

Based on the ECSE 4235 Library, here is how we can refine the software architecture:

- **Native Non-Blocking Input:** We can skip writing a custom `termios` configuration for the C state machine. The library provides `E4235_KYBdeblock(1)` to unblock standard input, meaning `scanf` or `read` calls will return immediately so the keyboard polling won't halt the motor's continuous run loop.
- **Direct Assembly Calls:** We can replace the `class_gpio_write` placeholder in the `motor_driver.s` draft with the exact `E4235_Write` function. Because the library already expects the first four arguments to be passed in registers `R0` through `R3`, our AAPCS-compliant assembly routine will link up flawlessly.
- **Hardware Setup:** In `main.c`, we must call `E4235_Init()` right away to handle the hardware memory mapping. We will then use `E4235_Select(GPIO, 1)` to configure the SN754410NE control pins and the 4N25 optocoupler pin as outputs.

Shall we draft the `main.c` top-level controller next to map out the Checkpoint 1 keyboard commands (enable/disable, CW/CCW, one revolution, run/stop, and faster/slower) using the `E4235_KYBdeblock` function?

User prompt: go ahead and make the edits onto the motor driver

Response:
```
.data
.align 2

@ Tracks the current position in the step sequence (0-7)
current_step:
    .word 0

@ Full-step coil patterns (4 steps: AB, BC, CD, DA)
@ Bits correspond to coils: 3=A, 2=B, 1=C, 0=D
full_step_table:
    .byte 0b1010, 0b0110, 0b0101, 0b1001

@ Half-step coil patterns (8 steps)
half_step_table:
    .byte 0b1000, 0b1010, 0b0010, 0b0110, 0b0100, 0b0101, 0b0001, 0b1001

.text
.global enable_routine
.global step_routine
.global step_timing_loop
.extern E4235_Write

@ ------------------------------------------------------------------------
@ enable_routine
@ Turns the H-bridge on or off through the 4N25 optocoupler[cite: 5, 8].
@ Args:
@   r0: Enable pin number
@   r1: State (1 = Enable, 0 = Disable)
@ ------------------------------------------------------------------------
enable_routine:
    push {lr}
    @ AAPCS: r0 (GPIO) and r1 (state) are passed directly to the library[cite: 46, 51]
    bl E4235_Write[cite: 46, 57]
    pop {pc}

@ ------------------------------------------------------------------------
@ step_routine
@ Looks up the next coil pattern and drives the SN754410NE inputs[cite: 8, 9].
```

```
@ Args:
@   r0: Direction (1 = CW, 0 = CCW)
@   r1: Mode (1 = Half step, 0 = Full step)
@   r2: Pointer to integer array containing the 4 motor pins {A, B, C, D}
@ ------------------------------------------------------------------------
step_routine:
    push {r4-r8, lr}

    mov r4, r0              @ Save direction
    mov r5, r1              @ Save mode
    mov r6, r2              @ Save pin array pointer pointer

    @ Load current step index
    ldr r7, =current_step
    ldr r8, [r7]

    @ Update step index based on direction
    cmp r4, #1
    addeq r8, r8, #1        @ If CW, increment
    subne r8, r8, #1        @ If CCW, decrement

    @ Handle wrapping based on mode
    cmp r5, #1
    beq half_step_wrap

full_step_wrap:
    and r8, r8, #3          @ Wrap 0-3 for full step (modulo 4)
    ldr r4, =full_step_table
    b fetch_pattern

half_step_wrap:
    and r8, r8, #7          @ Wrap 0-7 for half step (modulo 8)
    ldr r4, =half_step_table

fetch_pattern:
    @ Store updated step index
    str r8, [r7]

    @ Fetch the current bit pattern from the table
    ldrb r4, [r4, r8]       @ r4 now holds the 4-bit pattern

    @ Drive Pin A (Bit 3)
    ldr r0, [r6, #0]        @ Load Pin A number from array into r0[cite: 46, 51]
    and r1, r4, #0b1000
    lsr r1, r1, #3          @ Shift bit down to 0 or 1 for state in r1[cite: 46, 51]
    bl E4235_Write[cite: 46, 57]

    @ Drive Pin B (Bit 2)
    ldr r0, [r6, #4]
    and r1, r4, #0b0100
    lsr r1, r1, #2
    bl E4235_Write[cite: 46, 57]

    @ Drive Pin C (Bit 1)
    ldr r0, [r6, #8]
    and r1, r4, #0b0010
    lsr r1, r1, #1
    bl E4235_Write[cite: 46, 57]

    @ Drive Pin D (Bit 0)
    ldr r0, [r6, #12]
    and r1, r4, #0b0001
    bl E4235_Write[cite: 46, 57]

    pop {r4-r8, pc}

@ ------------------------------------------------------------------------
@ step_timing_loop
@ Waits until the next step is due using the ARM generic timer[cite: 8, 9].
@ Args:
@   r0: Target wait interval in timer ticks
@ ------------------------------------------------------------------------
step_timing_loop:
    push {r4, lr}
```

```
    mov r4, r0             @ Save target interval

    @ Read current 32-bit timer value (CNTPCT lower half) into r1
    mrc p15, 0, r1, c14, c0, 0
    add r4, r1, r4         @ Calculate target end time

wait_loop:
    mrc p15, 0, r1, c14, c0, 0
    cmp r1, r4
    blt wait_loop          @ Keep looping until current time >= end time

    pop {r4, pc}
```

The `enable_routine` and `step_routine` now declare `.extern E4235_Write` and branch directly to the library function `E4235_Write` to drive the GPIO outputs. The assembly registers `r0` and `r1` perfectly map to the `GPIO` and `state` arguments expected by the library. The `step_timing_loop` is preserved exactly as written to satisfy the project requirement of using the ARM generic timer directly from assembly.

User prompt: now lets move onto the next step

Response: The next logical step is to build the top-level C controller (`main.c`) to handle the Checkpoint 1 keyboard commands and state machine. This program will configure the GPIO pins, unblock the standard input using the class library, and continuously poll for keystrokes to control the motor.

Here is the complete `main.c` implementation that interfaces with your `motor_driver.s` assembly routines and the `E4235` library.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include "E4235.h" // Class library header

// External assembly functions from motor_driver.s
extern void enable_routine(int pin, int state);
extern void step_routine(int direction, int mode, int* pins);
extern void step_timing_loop(unsigned int ticks);

// Define GPIO pins (Adjust these to match your actual wiring)
#define PIN_ENABLE 17
#define PIN_A 18
#define PIN_B 19
#define PIN_C 20
#define PIN_D 21

// RP4 ARM Generic Timer frequency (typically 54 MHz)
#define TIMER_FREQ 54000000

int main() {
    // 1. Initialize Hardware and GPIO
    if (E4235_Init() != 0) {
        printf("Failed to initialize E4235 library.\n");
        return -1;
    }

    // Set motor and enable pins to OUTPUT (state = 1)
    E4235_Select(PIN_ENABLE, 1);
    int motor_pins[4] = {PIN_A, PIN_B, PIN_C, PIN_D};
    for (int i = 0; i < 4; i++) {
        E4235_Select(motor_pins[i], 1);
    }

    // 2. Initial State Variables
    int mode = 0;           // 0 = Full step, 1 = Half step
    int direction = 1;      // 1 = Clockwise, 0 = Counterclockwise
    int running = 0;        // 0 = Stopped, 1 = Continuous Run
    int enabled = 0;        // 0 = Disabled, 1 = Enabled
    float steps_per_sec = 100.0; // Initial step rate

    // Checkpoint 1 requirement: Print step mode at startup
    printf("PARMCO Motor Controller Initialized.\n");
    printf("Step Mode: %s\n", mode == 0 ? "Full Step (200/rev)" : "Half Step (400/rev)");
    printf("Controls: [e]nable, [d]irection, [o]ne rev, [r]un/stop, [f]aster, [s]lower, [q]uit\n");

    // 3. Unblock standard input for non-blocking keyboard polling
    E4235_KYBdeblock(1);
```

```
    char key = 0;
    while (key != 'q') {
        // Read keystroke without blocking
        if (read(STDIN_FILENO, &key, 1) > 0) {
            switch (key) {
                case 'e': // Enable/Disable H-Bridge
                    enabled = !enabled;
                    enable_routine(PIN_ENABLE, enabled);
                    printf("Motor %s.\n", enabled ? "ENABLED" : "DISABLED");
                    break;

                case 'd': // Toggle Direction
                    direction = !direction;
                    printf("Direction: %s\n", direction ? "Clockwise" : "Counterclockwise");
                    break;

                case 'o': // One Revolution
                    if (enabled) {
                        printf("Executing one revolution...\n");
                        int steps_needed = (mode == 0) ? 200 : 400;
                        unsigned int tick_delay = (unsigned int)(TIMER_FREQ / steps_per_sec);
                        for (int i = 0; i < steps_needed; i++) {
                            step_routine(direction, mode, motor_pins);
                            step_timing_loop(tick_delay);
                        }
                        running = 0; // Ensure we stop after one revolution
                        printf("Revolution complete.\n");
                    } else {
                        printf("Cannot step: Motor is disabled.\n");
                    }
                    break;

                case 'r': // Continuous Run / Stop
                    running = !running;
                    printf("Motor %s.\n", running ? "RUNNING" : "STOPPED");
                    break;

                case 'f': // Faster (increase by ~5%)
                    steps_per_sec *= 1.05;
                    printf("Speed increased. Current rate: %.2f steps/sec\n", steps_per_sec);
                    break;

                case 's': // Slower (decrease by ~5%)
                    steps_per_sec *= 0.95;
                    if (steps_per_sec < 10.0) steps_per_sec = 10.0; // Prevent stalling at zero
                    printf("Speed decreased. Current rate: %.2f steps/sec\n", steps_per_sec);
                    break;
            }
        }

        // 4. Continuous Running Logic
        if (running && enabled) {
            unsigned int tick_delay = (unsigned int)(TIMER_FREQ / steps_per_sec);
            step_routine(direction, mode, motor_pins);
            step_timing_loop(tick_delay);
        } else {
            // Sleep briefly to prevent 100% CPU usage when idle
            usleep(10000);
        }
    }

    // 5. Cleanup before exit
    running = 0;
    enable_routine(PIN_ENABLE, 0);
    E4235_KYBdeblock(0); // Restore blocking IO
    printf("Exiting PARMCO.\n");
    return 0;
}
```

This controller handles the non-blocking terminal input while maintaining the continuous timing loop necessary for smooth stepper operation. If you are ready, we can draft the `Makefile` to link `main.c`, `motor_driver.s`, and `libE4235.a` together, or we can move right into drafting the Checkpoint 1 pre-power inspection document.

User prompt: lets make some changes to the keys. Lets use E to enable/disable the motor. C can be used for direction of rotation. The spacebar for one revolution. S for start/stop. Arrow up and down for increase or decrease speed

Response: To handle the multi-byte escape sequences generated by the arrow keys, the non-blocking `read` must be adjusted to pull up to three bytes at a time.

Here is the updated `main.c` with the new key mappings.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include "E4235.h" // Class library header[cite: 46]

// External assembly functions from motor_driver.s
extern void enable_routine(int pin, int state);
extern void step_routine(int direction, int mode, int* pins);
extern void step_timing_loop(unsigned int ticks);

// Define GPIO pins (Adjust these to match your actual wiring)
#define PIN_ENABLE 17
#define PIN_A 18
#define PIN_B 19
#define PIN_C 20
#define PIN_D 21

// RP4 ARM Generic Timer frequency (typically 54 MHz)
#define TIMER_FREQ 54000000

int main() {
    // 1. Initialize Hardware and GPIO
    if (E4235_Init() != 0) { //[cite: 46, 56]
        printf("Failed to initialize E4235 library.\n");
        return -1;
    }

    // Set motor and enable pins to OUTPUT (state = 1)
    E4235_Select(PIN_ENABLE, 1); //[cite: 46, 56, 57]
    int motor_pins[4] = {PIN_A, PIN_B, PIN_C, PIN_D};
    for (int i = 0; i < 4; i++) {
        E4235_Select(motor_pins[i], 1); //[cite: 46, 56, 57]
    }

    // 2. Initial State Variables
    int mode = 0;           // 0 = Full step, 1 = Half step
    int direction = 1;      // 1 = Clockwise, 0 = Counterclockwise
    int running = 0;        // 0 = Stopped, 1 = Continuous Run
    int enabled = 0;        // 0 = Disabled, 1 = Enabled
    float steps_per_sec = 100.0; // Initial step rate

    // Checkpoint 1 requirement: Print step mode at startup[cite: 7]
    printf("PARMCO Motor Controller Initialized.\n");
    printf("Step Mode: %s\n", mode == 0 ? "Full Step (200/rev)" : "Half Step (400/rev)");
    printf("Controls: [E]nable, [C]irection, [Space] One rev, [S]tart/Stop, [Up] Faster, [Down] Slower, [Q]uit

    // 3. Unblock standard input for non-blocking keyboard polling
    E4235_KYBdeblock(1); //[cite: 46, 52, 53]

    char buf[3];
    int run_program = 1;

    while (run_program) {
        // Read up to 3 bytes to catch arrow key escape sequences
        int bytes_read = read(STDIN_FILENO, buf, 3);

        if (bytes_read > 0) {
            if (bytes_read == 1) {
                switch (buf[0]) {
                    case 'e':
                    case 'E': // Enable/Disable H-Bridge
                        enabled = !enabled;
                        enable_routine(PIN_ENABLE, enabled);
                        printf("Motor %s.\n", enabled ? "ENABLED" : "DISABLED");
                        break;
```

```
                    case 'c':
                    case 'C': // Toggle Direction
                        direction = !direction;
                        printf("Direction: %s\n", direction ? "Clockwise" : "Counterclockwise");
                        break;

                    case ' ': // Spacebar: One Revolution
                        if (enabled) {
                            printf("Executing one revolution...\n");
                            int steps_needed = (mode == 0) ? 200 : 400; //[cite: 7]
                            unsigned int tick_delay = (unsigned int)(TIMER_FREQ / steps_per_sec);
                            for (int i = 0; i < steps_needed; i++) {
                                step_routine(direction, mode, motor_pins);
                                step_timing_loop(tick_delay);
                            }
                            running = 0; // Ensure we stop after one revolution
                            printf("Revolution complete.\n");
                        } else {
                            printf("Cannot step: Motor is disabled.\n");
                        }
                        break;

                    case 's':
                    case 'S': // Continuous Run / Stop
                        running = !running;
                        printf("Motor %s.\n", running ? "RUNNING" : "STOPPED");
                        break;

                    case 'q':
                    case 'Q': // Quit
                        run_program = 0;
                        break;
                }
            } else if (bytes_read == 3 && buf[0] == '\033' && buf[1] == '[') {
                // Arrow keys generate a 3-byte sequence: ESC (0x1B), '[', and 'A', 'B', 'C', or 'D'
                if (buf[2] == 'A') { // Up Arrow: Faster (increase by ~5%)[cite: 7]
                    steps_per_sec *= 1.05;
                    printf("Speed increased. Current rate: %.2f steps/sec\n", steps_per_sec);
                } else if (buf[2] == 'B') { // Down Arrow: Slower (decrease by ~5%)[cite: 7]
                    steps_per_sec *= 0.95;
                    if (steps_per_sec < 10.0) steps_per_sec = 10.0; // Prevent stalling at zero
                    printf("Speed decreased. Current rate: %.2f steps/sec\n", steps_per_sec);
                }
            }
        }

        // 4. Continuous Running Logic
        if (running && enabled) {
            unsigned int tick_delay = (unsigned int)(TIMER_FREQ / steps_per_sec);
            step_routine(direction, mode, motor_pins);
            step_timing_loop(tick_delay);
        } else {
            // Sleep briefly to prevent 100% CPU usage when idle
            usleep(10000);
        }
    }

    // 5. Cleanup before exit
    running = 0;
    enable_routine(PIN_ENABLE, 0);
    E4235_KYBdeblock(0); // Restore blocking IO[cite: 46, 52, 53]
    printf("Exiting PARMCO.\n");
    return 0;
}
```

User prompt: we also need a make file

Response: To manage the build process within your VS Code SSH environment, this `Makefile` links the C controller and ARM assembly driver with the class library. The library documentation specifies using the `-lE4235` flag during compilation.

```
# Compiler and flags
CC = gcc
CFLAGS = -Wall -g
LDFLAGS = -lE4235

# Target executable name
TARGET = parmco_cp1

# Object files
OBJS = main.o motor_driver.o

# Default target
all: $(TARGET)

# Link the object files and the ECSE 4235 library
$(TARGET): $(OBJS)
	$(CC) $(CFLAGS) -o $(TARGET) $(OBJS) $(LDFLAGS)

# Compile the C top-level controller
main.o: main.c
	$(CC) $(CFLAGS) -c main.c

# Compile the ARM assembly motor driver
motor_driver.o: motor_driver.s
	$(CC) $(CFLAGS) -c motor_driver.s

# Clean up build artifacts
clean:
	rm -f $(OBJS) $(TARGET)
```

Are there any specific file naming conventions your team prefers for the target executable or object files before you run the hardware test?

User prompt: the pins labeled as A-D, can you specify which pins they are?

Response: Pins A, B, C, and D logically represent the four control inputs to the SN754410NE H-bridge driver, which energize the two coils of your Nema17 stepper motor.

Because the project guidelines state that your team generates the hardware schematic, the exact Raspberry Pi 4 GPIO numbers for these pins are not locked by the project requirements. They are simply whichever four standard GPIO pins your lab partner chooses to wire to the SN754410NE's logic inputs (pins 1A, 2A, 3A, and 4A on the IC).

Since your lab partner is currently building the Checkpoint 1 hardware circuit, you will need to ask them which four RP4 GPIO pins they selected. Once they provide those numbers, you can update the `#define PIN_A`, `PIN_B`, `PIN_C`, and `PIN_D` macros at the top of your `main.c` file to match their wiring exactly.

User prompt: after a series of tests with the code, we made some minor pin changes to the code and tried to run the program. It seems like the user input is not being read correctly. could we attempt to remove the arrows keys and replace them with u and d for up and down speed?

Response: Reading multi-byte escape sequences for arrow keys in a non-blocking terminal can fragment the input buffer, causing standard keystrokes to be missed or misread. Reverting to single-character inputs simplifies the read operation to a single byte at a time, ensuring reliable polling.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include "E4235.h" // Class library header[cite: 46]

// External assembly functions from motor_driver.s
extern void enable_routine(int pin, int state);
extern void step_routine(int direction, int mode, int* pins);
extern void step_timing_loop(unsigned int ticks);

// Define GPIO pins (Update these to match your lab partner's exact wiring)
#define PIN_ENABLE 17
#define PIN_A 18
#define PIN_B 19
#define PIN_C 20
#define PIN_D 21

// RP4 ARM Generic Timer frequency (typically 54 MHz)
#define TIMER_FREQ 54000000
```

```
int main() {
    // 1. Initialize Hardware and GPIO
    if (E4235_Init() != 0) { //[cite: 46, 56]
        printf("Failed to initialize E4235 library.\n");
        return -1;
    }

    // Set motor and enable pins to OUTPUT (state = 1)
    E4235_Select(PIN_ENABLE, 1); //[cite: 46, 56, 57]
    int motor_pins[4] = {PIN_A, PIN_B, PIN_C, PIN_D};
    for (int i = 0; i < 4; i++) {
        E4235_Select(motor_pins[i], 1); //[cite: 46, 56, 57]
    }

    // 2. Initial State Variables
    int mode = 0;           // 0 = Full step, 1 = Half step
    int direction = 1;      // 1 = Clockwise, 0 = Counterclockwise
    int running = 0;        // 0 = Stopped, 1 = Continuous Run
    int enabled = 0;        // 0 = Disabled, 1 = Enabled
    float steps_per_sec = 100.0; // Initial step rate

    // Checkpoint 1 requirement: Print step mode at startup[cite: 7]
    printf("PARMCO Motor Controller Initialized.\n");
    printf("Step Mode: %s\n", mode == 0 ? "Full Step (200/rev)" : "Half Step (400/rev)");
    printf("Controls: [E]nable, [C]irection, [Space] One rev, [S]tart/Stop, [U] Faster, [D] Slower, [Q]uit\n");

    // 3. Unblock standard input for non-blocking keyboard polling
    E4235_KYBdeblock(1); //[cite: 46, 52, 53]

    char key = 0;
    int run_program = 1;

    while (run_program) {
        // Read strictly 1 byte at a time to prevent buffering issues
        if (read(STDIN_FILENO, &key, 1) > 0) {
            switch (key) {
                case 'e':
                case 'E': // Enable/Disable H-Bridge
                    enabled = !enabled;
                    enable_routine(PIN_ENABLE, enabled);
                    printf("Motor %s.\n", enabled ? "ENABLED" : "DISABLED");
                    break;

                case 'c':
                case 'C': // Toggle Direction
                    direction = !direction;
                    printf("Direction: %s\n", direction ? "Clockwise" : "Counterclockwise");
                    break;

                case ' ': // Spacebar: One Revolution
                    if (enabled) {
                        printf("Executing one revolution...\n");
                        int steps_needed = (mode == 0) ? 200 : 400; //[cite: 7]
                        unsigned int tick_delay = (unsigned int)(TIMER_FREQ / steps_per_sec);
                        for (int i = 0; i < steps_needed; i++) {
                            step_routine(direction, mode, motor_pins);
                            step_timing_loop(tick_delay);
                        }
                        running = 0; // Ensure we stop after one revolution
                        printf("Revolution complete.\n");
                    } else {
                        printf("Cannot step: Motor is disabled.\n");
                    }
                    break;

                case 's':
                case 'S': // Continuous Run / Stop
                    running = !running;
                    printf("Motor %s.\n", running ? "RUNNING" : "STOPPED");
                    break;

                case 'u':
                case 'U': // Faster (increase by ~5%)[cite: 7]
                    steps_per_sec *= 1.05;
```

```
                    printf("Speed increased. Current rate: %.2f steps/sec\n", steps_per_sec);
                    break;

                case 'd':
                case 'D': // Slower (decrease by ~5%)[cite: 7]
                    steps_per_sec *= 0.95;
                    if (steps_per_sec < 10.0) steps_per_sec = 10.0; // Prevent stalling at zero
                    printf("Speed decreased. Current rate: %.2f steps/sec\n", steps_per_sec);
                    break;

                case 'q':
                case 'Q': // Quit
                    run_program = 0;
                    break;
            }
            // Clear the key variable so it doesn't repeatedly trigger if read() returns 0 next loop
            key = 0;
        }

        // 4. Continuous Running Logic
        if (running && enabled) {
            unsigned int tick_delay = (unsigned int)(TIMER_FREQ / steps_per_sec);
            step_routine(direction, mode, motor_pins);
            step_timing_loop(tick_delay);
        } else {
            // Sleep briefly to prevent 100% CPU usage when idle
            usleep(10000);
        }
    }

    // 5. Cleanup before exit
    running = 0;
    enable_routine(PIN_ENABLE, 0);
    E4235_KYBdeblock(0); // Restore blocking IO[cite: 46, 52, 53]
    printf("Exiting PARMCO.\n");
    return 0;
}
```

User prompt: new bug. keystroke inputs now mostly work. however, when the motor is enabled then disabled, then enabled again then type an S, the keystroke is not registered

Response: This is a classic symptom of a non-blocking I/O buffer bottleneck.

Because `E4235_KYBdeblock(1)` sets the standard input to return immediately rather than waiting for a full line, the OS is capturing every single byte you generate—including hidden newline characters (`\n` or `\r`) if you are pressing Enter, or auto-repeated characters if a key is held down a fraction of a second too long.

Right now, the code uses `if (read(STDIN_FILENO, &key, 1) > 0)`. This only pulls **one byte** out of the buffer per loop iteration (which is every 10ms). If you rapidly toggle the motor (e.g., typing `E`, `E`, `E`), you are queuing up multiple bytes. The loop falls behind, and your `S` keystroke gets buried in a backlog of unprocessed bytes, making it seem like it wasn't registered.

To fix this, we need to change the `if` statement to a `while` loop. This instantly drains the entire input buffer on every cycle before moving on to the motor timing logic, ensuring no keystrokes are left pending.

Here is the updated `main.c` with the buffer-draining loop and a filter to ignore newline characters:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include "E4235.h" // Class library header[cite: 46]

// External assembly functions from motor_driver.s
extern void enable_routine(int pin, int state);
extern void step_routine(int direction, int mode, int* pins);
extern void step_timing_loop(unsigned int ticks);

// Define GPIO pins (Update these to match your lab partner's exact wiring)
#define PIN_ENABLE 17
#define PIN_A 18
#define PIN_B 19
#define PIN_C 20
#define PIN_D 21
```

```
// RP4 ARM Generic Timer frequency (typically 54 MHz)
#define TIMER_FREQ 54000000

int main() {
    // 1. Initialize Hardware and GPIO
    if (E4235_Init() != 0) { //[cite: 46, 56]
        printf("Failed to initialize E4235 library.\n");
        return -1;
    }

    // Set motor and enable pins to OUTPUT (state = 1)
    E4235_Select(PIN_ENABLE, 1); //[cite: 46, 56, 57]
    int motor_pins[4] = {PIN_A, PIN_B, PIN_C, PIN_D};
    for (int i = 0; i < 4; i++) {
        E4235_Select(motor_pins[i], 1); //[cite: 46, 56, 57]
    }

    // 2. Initial State Variables
    int mode = 0;           // 0 = Full step, 1 = Half step
    int direction = 1;      // 1 = Clockwise, 0 = Counterclockwise
    int running = 0;        // 0 = Stopped, 1 = Continuous Run
    int enabled = 0;        // 0 = Disabled, 1 = Enabled
    float steps_per_sec = 100.0; // Initial step rate

    // Checkpoint 1 requirement: Print step mode at startup[cite: 7]
    printf("PARMCO Motor Controller Initialized.\n");
    printf("Step Mode: %s\n", mode == 0 ? "Full Step (200/rev)" : "Half Step (400/rev)");
    printf("Controls: [E]nable, [C]irection, [Space] One rev, [S]tart/Stop, [U] Faster, [D] Slower, [Q]uit\n")

    // 3. Unblock standard input for non-blocking keyboard polling
    E4235_KYBdeblock(1); //[cite: 46, 52, 53]

    char key = 0;
    int run_program = 1;

    while (run_program) {
        // Read strictly 1 byte at a time in a WHILE loop to instantly drain the buffer
        while (read(STDIN_FILENO, &key, 1) > 0) {

            // Ignore trailing newlines or carriage returns that might be in the buffer
            if (key == '\n' || key == '\r') {
                continue;
            }

            switch (key) {
                case 'e':
                case 'E': // Enable/Disable H-Bridge
                    enabled = !enabled;
                    enable_routine(PIN_ENABLE, enabled);
                    printf("Motor %s.\n", enabled ? "ENABLED" : "DISABLED");
                    break;

                case 'c':
                case 'C': // Toggle Direction
                    direction = !direction;
                    printf("Direction: %s\n", direction ? "Clockwise" : "Counterclockwise");
                    break;

                case ' ': // Spacebar: One Revolution
                    if (enabled) {
                        printf("Executing one revolution...\n");
                        int steps_needed = (mode == 0) ? 200 : 400; //[cite: 7]
                        unsigned int tick_delay = (unsigned int)(TIMER_FREQ / steps_per_sec);
                        for (int i = 0; i < steps_needed; i++) {
                            step_routine(direction, mode, motor_pins);
                            step_timing_loop(tick_delay);
                        }
                        running = 0; // Ensure we stop after one revolution
                        printf("Revolution complete.\n");
                    } else {
                        printf("Cannot step: Motor is disabled.\n");
                    }
                    break;
```

```
                case 's':
                case 'S': // Continuous Run / Stop
                    running = !running;
                    printf("Motor %s.\n", running ? "RUNNING" : "STOPPED");
                    break;

                case 'u':
                case 'U': // Faster (increase by ~5%)[cite: 7]
                    steps_per_sec *= 1.05;
                    printf("Speed increased. Current rate: %.2f steps/sec\n", steps_per_sec);
                    break;

                case 'd':
                case 'D': // Slower (decrease by ~5%)[cite: 7]
                    steps_per_sec *= 0.95;
                    if (steps_per_sec < 10.0) steps_per_sec = 10.0; // Prevent stalling at zero
                    printf("Speed decreased. Current rate: %.2f steps/sec\n", steps_per_sec);
                    break;

                case 'q':
                case 'Q': // Quit
                    run_program = 0;
                    break;
            }
        }

        // 4. Continuous Running Logic
        if (running && enabled) {
            unsigned int tick_delay = (unsigned int)(TIMER_FREQ / steps_per_sec);
            step_routine(direction, mode, motor_pins);
            step_timing_loop(tick_delay);
        } else {
            // Sleep briefly to prevent 100% CPU usage when idle
            usleep(10000);
        }
    }

    // 5. Cleanup before exit
    running = 0;
    enable_routine(PIN_ENABLE, 0);
    E4235_KYBdeblock(0); // Restore blocking IO[cite: 46, 52, 53]
    printf("Exiting PARMCO.\n");
    return 0;
}
```

User prompt: before i copy and paste the code, the enable pin also remains at its current state, even if i quit the program. we need to make sure the enable pin only remains enabled during run time

Response: To ensure the enable pin is safely disabled no matter how the program terminates (including force-quits via `Ctrl+C`), we need to add a standard POSIX signal handler. Additionally, the project hardware rules state that the motor is disabled when the GPIO is low or floating. Reverting the GPIO pins to inputs (read mode) during cleanup guarantees they float, creating a hardware-level fail-safe.

Here is the updated `main.c` with the signal handler and the enhanced hardware cleanup routine:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include "E4235.h" // Class library header[cite: 46]

// External assembly functions from motor_driver.s
extern void enable_routine(int pin, int state);
extern void step_routine(int direction, int mode, int* pins);
extern void step_timing_loop(unsigned int ticks);

// Define GPIO pins (Update these to match your lab partner's exact wiring)
#define PIN_ENABLE 17
#define PIN_A 18
#define PIN_B 19
#define PIN_C 20
#define PIN_D 21

// RP4 ARM Generic Timer frequency (typically 54 MHz)
```

```
#define TIMER_FREQ 54000000

// Global flag for the main loop, allowing the signal handler to gracefully exit
volatile int run_program = 1;

// Signal handler to catch Ctrl+C and ensure cleanup executes
void handle_sigint(int sig) {
    run_program = 0;
}

int main() {
    // Register the signal handler for SIGINT (Ctrl+C)
    signal(SIGINT, handle_sigint);

    // 1. Initialize Hardware and GPIO
    if (E4235_Init() != 0) { //[cite: 46, 56]
        printf("Failed to initialize E4235 library.\n");
        return -1;
    }

    // Set motor and enable pins to OUTPUT (state = 1)
    E4235_Select(PIN_ENABLE, 1); //[cite: 46, 56, 57]
    int motor_pins[4] = {PIN_A, PIN_B, PIN_C, PIN_D};
    for (int i = 0; i < 4; i++) {
        E4235_Select(motor_pins[i], 1); //[cite: 46, 56, 57]
    }

    // 2. Initial State Variables
    int mode = 0;           // 0 = Full step, 1 = Half step
    int direction = 1;      // 1 = Clockwise, 0 = Counterclockwise
    int running = 0;        // 0 = Stopped, 1 = Continuous Run
    int enabled = 0;        // 0 = Disabled, 1 = Enabled
    float steps_per_sec = 100.0; // Initial step rate

    // Checkpoint 1 requirement: Print step mode at startup[cite: 7]
    printf("PARMCO Motor Controller Initialized.\n");
    printf("Step Mode: %s\n", mode == 0 ? "Full Step (200/rev)" : "Half Step (400/rev)");
    printf("Controls: [E]nable, [C]irection, [Space] One rev, [S]tart/Stop, [U] Faster, [D] Slower, [Q]uit\n")

    // 3. Unblock standard input for non-blocking keyboard polling
    E4235_KYBdeblock(1); //[cite: 46, 52, 53]

    char key = 0;

    while (run_program) {
        // Read strictly 1 byte at a time in a WHILE loop to instantly drain the buffer
        while (read(STDIN_FILENO, &key, 1) > 0) {

            // Ignore trailing newlines or carriage returns
            if (key == '\n' || key == '\r') {
                continue;
            }

            switch (key) {
                case 'e':
                case 'E': // Enable/Disable H-Bridge
                    enabled = !enabled;
                    enable_routine(PIN_ENABLE, enabled);
                    printf("Motor %s.\n", enabled ? "ENABLED" : "DISABLED");
                    break;

                case 'c':
                case 'C': // Toggle Direction
                    direction = !direction;
                    printf("Direction: %s\n", direction ? "Clockwise" : "Counterclockwise");
                    break;

                case ' ': // Spacebar: One Revolution
                    if (enabled) {
                        printf("Executing one revolution...\n");
                        int steps_needed = (mode == 0) ? 200 : 400; //[cite: 7]
                        unsigned int tick_delay = (unsigned int)(TIMER_FREQ / steps_per_sec);
                        for (int i = 0; i < steps_needed; i++) {
                            step_routine(direction, mode, motor_pins);
```

```
                            step_timing_loop(tick_delay);
                        }
                        running = 0; // Ensure we stop after one revolution
                        printf("Revolution complete.\n");
                    } else {
                        printf("Cannot step: Motor is disabled.\n");
                    }
                    break;

                case 's':
                case 'S': // Continuous Run / Stop
                    running = !running;
                    printf("Motor %s.\n", running ? "RUNNING" : "STOPPED");
                    break;

                case 'u':
                case 'U': // Faster (increase by ~5%)[cite: 7]
                    steps_per_sec *= 1.05;
                    printf("Speed increased. Current rate: %.2f steps/sec\n", steps_per_sec);
                    break;

                case 'd':
                case 'D': // Slower (decrease by ~5%)[cite: 7]
                    steps_per_sec *= 0.95;
                    if (steps_per_sec < 10.0) steps_per_sec = 10.0; // Prevent stalling at zero
                    printf("Speed decreased. Current rate: %.2f steps/sec\n", steps_per_sec);
                    break;

                case 'q':
                case 'Q': // Quit
                    run_program = 0;
                    break;
            }
            key = 0;
        }

        // 4. Continuous Running Logic
        if (running && enabled) {
            unsigned int tick_delay = (unsigned int)(TIMER_FREQ / steps_per_sec);
            step_routine(direction, mode, motor_pins);
            step_timing_loop(tick_delay);
        } else {
            // Sleep briefly to prevent 100% CPU usage when idle
            usleep(10000);
        }
    }

    // 5. Cleanup before exit
    // Drive the 4N25 optocoupler low and set to input so it floats[cite: 8]
    enable_routine(PIN_ENABLE, 0);
    E4235_Select(PIN_ENABLE, 0); //[cite: 46, 56, 57]

    // De-energize and float the motor coils
    for (int i = 0; i < 4; i++) {
        E4235_Write(motor_pins[i], 0); //[cite: 46, 57]
        E4235_Select(motor_pins[i], 0); //[cite: 46, 56, 57]
    }

    E4235_KYBdeblock(0); // Restore blocking IO[cite: 46, 52, 53]
    printf("\nExiting PARMCO safely.\n");
    return 0;
}
```

User prompt: #include <stdio.h> #include <stdlib.h> #include <unistd.h> #include "E4235.h" // Class library header[cite: 46] // External assembly functions from motor_driver.s extern void enable_routine(int pin, int state); extern void step_routine(int direction, int mode, int pins); extern void step_timing_loop(unsigned int ticks); // Define GPIO pins (Update these to match your lab partner's exact wiring) #define PIN_ENABLE 24 #define PIN_A 17 #define PIN_B 27 #define PIN_C 22 #define PIN_D 23 // RP4 ARM Generic Timer frequency (typically 54 MHz) #define TIMER_FREQ 54000000 void test_motor_gpio(void) { int pins[4] = {17, 27, 22, 23}; int driver_pins[4] = {2, 7, 10, 15}; printf("\nStarting motor GPIO test...\n"); // Keep the H-bridge disabled during testing E4235_Write(PIN_ENABLE, 0); // Initialize all motor GPIOs LOW for (int i = 0; i < 4; i++) { E4235_Write(pins[i], 0); } for (int i = 0; i < 4; i++) { // Set all motor GPIOs LOW for (int j = 0; j < 4; j++) { E4235_Write(pins[j], 0); } // Set the GPIO being tested HIGH E4235_Write(pins[i], 1); printf("\nGPIO%d is HIGH.\n", pins[i]); printf("Measure SN754410NE pin %d.\n", driver_pins[i]); printf("Expected voltage: approximately 3.3 V\n"); printf("Press Enter when finished measuring...\n"); getchar(); } // Return all GPIOs LOW after testing for (int i = 0; i < 4; i++) { E4235_Write(pins[i], 0); } printf("\nAll GPIO tests completed.\n"); } int main() { // 1. Initialize Hardware and GPIO if (E4235_Init() != 0) { //[cite: 46, 56]*

*printf("Failed to initialize E4235 library.\n"); return -1; } // Set motor and enable pins to OUTPUT (state = 1) E4235_Select(PIN_ENABLE, 1); //[cite: 46, 56, 57] int motor_pins[4] = {PIN_A, PIN_B, PIN_C, PIN_D}; for (int i = 0; i < 4; i++) { E4235_Select(motor_pins[i], 1); //[cite: 46, 56, 57] } // 2. Initial State Variables int mode = 0; // 0 = Full step, 1 = Half step int direction = 1; // 1 = Clockwise, 0 = Counterclockwise int running = 0; // 0 = Stopped, 1 = Continuous Run int enabled = 0; // 0 = Disabled, 1 = Enabled float steps_per_sec = 100.0; // Initial step rate // Checkpoint 1 requirement: Print step mode at startup[cite: 7] printf("PARMCO Motor Controller Initialized.\n"); printf("Step Mode: %s\n", mode == 0 ? "Full Step (200/rev)" : "Half Step (400/rev)"); printf("Controls: [E]nable, [C]irection, [Space] One rev, [S]tart/Stop, [U] Faster, [D] Slower, [Q]uit\n"); test_motor_gpio(); // 3. Unblock standard input for non-blocking keyboard polling E4235_KYBdeblock(1); //[cite: 46, 52, 53] char key = 0; int run_program = 1; while (run_program) { // Read strictly 1 byte at a time to prevent buffering issues if (read(STDIN_FILENO, &key, 1) > 0) { switch (key) { case 'e': case 'E': // Enable/Disable H-Bridge enabled = !enabled; enable_routine(PIN_ENABLE, enabled); printf("Motor %s.\n", enabled ? "ENABLED" : "DISABLED"); break; case 'c': case 'C': // Toggle Direction direction = !direction; printf("Direction: %s\n", direction ? "Clockwise" : "Counterclockwise"); break; case ' ': // Spacebar: One Revolution if (enabled) { printf("Executing one revolution...\n"); int steps_needed = (mode == 0) ? 200 : 400; //[cite: 7] unsigned int tick_delay = (unsigned int)(TIMER_FREQ / steps_per_sec); for (int i = 0; i < steps_needed; i++) { step_routine(direction, mode, motor_pins); step_timing_loop(tick_delay); } running = 0; // Ensure we stop after one revolution printf("Revolution complete.\n"); } else { printf("Cannot step: Motor is disabled.\n"); } break; case 's': case 'S': // Continuous Run / Stop running = !running; printf("Motor %s.\n", running ? "RUNNING" : "STOPPED"); break; case 'u': case 'U': // Faster (increase by ~5%)[cite: 7] steps_per_sec *= 1.05; printf("Speed increased. Current rate: %.2f steps/sec\n", steps_per_sec); break; case 'd': case 'D': // Slower (decrease by ~5%)[cite: 7] steps_per_sec *= 0.95; if (steps_per_sec < 10.0) steps_per_sec = 10.0; // Prevent stalling at zero printf("Speed decreased. Current rate: %.2f steps/sec\n", steps_per_sec); break; case 'q': case 'Q': // Quit run_program = 0; break; } // Clear the key variable so it doesn't repeatedly trigger if read() returns 0 next loop key = 0; } // 4. Continuous Running Logic if (running && enabled) { unsigned int tick_delay = (unsigned int)(TIMER_FREQ / steps_per_sec); step_routine(direction, mode, motor_pins); step_timing_loop(tick_delay); } else { // Sleep briefly to prevent 100% CPU usage when idle usleep(10000); } } // 5. Cleanup before exit running = 0; enable_routine(PIN_ENABLE, 0); E4235_KYBdeblock(0); // Restore blocking IO[cite: 46, 52, 53] printf("Exiting PARMCO.\n"); return 0; } edit this code to include the changes*

Response: Here is the updated `main.c` that merges your `test_motor_gpio()` function and new pin assignments with the buffer-draining `while` loop, the `SIGINT` (Ctrl+C) handler, and the hardware-level cleanup routine:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include "E4235.h" // Class library header[cite: 46]

// External assembly functions from motor_driver.s
extern void enable_routine(int pin, int state);
extern void step_routine(int direction, int mode, int* pins);
extern void step_timing_loop(unsigned int ticks);

// Define GPIO pins (Updated to match actual wiring)
#define PIN_ENABLE 24
#define PIN_A 17
#define PIN_B 27
#define PIN_C 22
#define PIN_D 23

// RP4 ARM Generic Timer frequency (typically 54 MHz)
#define TIMER_FREQ 54000000

// Global flag for the main loop, allowing the signal handler to gracefully exit
volatile int run_program = 1;

// Signal handler to catch Ctrl+C and ensure cleanup executes
void handle_sigint(int sig) {
    run_program = 0;
}

void test_motor_gpio(void) {
    int pins[4] = {17, 27, 22, 23};
    int driver_pins[4] = {2, 7, 10, 15};

    printf("\nStarting motor GPIO test...\n");

    // Keep the H-bridge disabled during testing
    E4235_Write(PIN_ENABLE, 0);[cite: 46, 57]

    // Initialize all motor GPIOs LOW
    for (int i = 0; i < 4; i++) {
        E4235_Write(pins[i], 0);[cite: 46, 57]
    }

    for (int i = 0; i < 4; i++) {

        // Set all motor GPIOs LOW
        for (int j = 0; j < 4; j++) {
            E4235_Write(pins[j], 0);[cite: 46, 57]
        }
```

```
            // Set the GPIO being tested HIGH
            E4235_Write(pins[i], 1);[cite: 46, 57]

            printf("\nGPIO%d is HIGH.\n", pins[i]);
            printf("Measure SN754410NE pin %d.\n", driver_pins[i]);
            printf("Expected voltage: approximately 3.3 V\n");
            printf("Press Enter when finished measuring...\n");

            getchar();
        }

        // Return all GPIOs LOW after testing
        for (int i = 0; i < 4; i++) {
            E4235_Write(pins[i], 0);[cite: 46, 57]
        }

        printf("\nAll GPIO tests completed.\n");
    }

    int main() {
        // Register the signal handler for SIGINT (Ctrl+C)
        signal(SIGINT, handle_sigint);

        // 1. Initialize Hardware and GPIO
        if (E4235_Init() != 0) {[cite: 46, 56]
            printf("Failed to initialize E4235 library.\n");
            return -1;
        }

        // Set motor and enable pins to OUTPUT (state = 1)
        E4235_Select(PIN_ENABLE, 1);[cite: 46, 56, 57]
        int motor_pins[4] = {PIN_A, PIN_B, PIN_C, PIN_D};
        for (int i = 0; i < 4; i++) {
            E4235_Select(motor_pins[i], 1);[cite: 46, 56, 57]
        }

        // 2. Initial State Variables
        int mode = 0;           // 0 = Full step, 1 = Half step
        int direction = 1;      // 1 = Clockwise, 0 = Counterclockwise
        int running = 0;        // 0 = Stopped, 1 = Continuous Run
        int enabled = 0;        // 0 = Disabled, 1 = Enabled
        float steps_per_sec = 100.0; // Initial step rate

        // Checkpoint 1 requirement: Print step mode at startup[cite: 7]
        printf("PARMCO Motor Controller Initialized.\n");
        printf("Step Mode: %s\n", mode == 0 ? "Full Step (200/rev)" : "Half Step (400/rev)");
        printf("Controls: [E]nable, [C]irection, [Space] One rev, [S]tart/Stop, [U] Faster, [D] Slower, [Q]uit\n")

        // Run the user-defined GPIO hardware test before unblocking the keyboard
        test_motor_gpio();

        // 3. Unblock standard input for non-blocking keyboard polling
        E4235_KYBdeblock(1);[cite: 46, 52, 53]

        char key = 0;

        while (run_program) {
            // Read strictly 1 byte at a time in a WHILE loop to instantly drain the buffer
            while (read(STDIN_FILENO, &key, 1) > 0) {

                // Ignore trailing newlines or carriage returns
                if (key == '\n' || key == '\r') {
                    continue;
                }

                switch (key) {
                    case 'e':
                    case 'E': // Enable/Disable H-Bridge
                        enabled = !enabled;
                        enable_routine(PIN_ENABLE, enabled);
                        printf("Motor %s.\n", enabled ? "ENABLED" : "DISABLED");
                        break;
```

```
            case 'c':
            case 'C': // Toggle Direction
                direction = !direction;
                printf("Direction: %s\n", direction ? "Clockwise" : "Counterclockwise");
                break;

            case ' ': // Spacebar: One Revolution
                if (enabled) {
                    printf("Executing one revolution...\n");
                    int steps_needed = (mode == 0) ? 200 : 400;[cite: 7]
                    unsigned int tick_delay = (unsigned int)(TIMER_FREQ / steps_per_sec);
                    for (int i = 0; i < steps_needed; i++) {
                        step_routine(direction, mode, motor_pins);
                        step_timing_loop(tick_delay);
                    }
                    running = 0; // Ensure we stop after one revolution
                    printf("Revolution complete.\n");
                } else {
                    printf("Cannot step: Motor is disabled.\n");
                }
                break;

            case 's':
            case 'S': // Continuous Run / Stop
                running = !running;
                printf("Motor %s.\n", running ? "RUNNING" : "STOPPED");
                break;

            case 'u':
            case 'U': // Faster (increase by ~5%)[cite: 7]
                steps_per_sec *= 1.05;
                printf("Speed increased. Current rate: %.2f steps/sec\n", steps_per_sec);
                break;

            case 'd':
            case 'D': // Slower (decrease by ~5%)[cite: 7]
                steps_per_sec *= 0.95;
                if (steps_per_sec < 10.0) steps_per_sec = 10.0; // Prevent stalling at zero
                printf("Speed decreased. Current rate: %.2f steps/sec\n", steps_per_sec);
                break;

            case 'q':
            case 'Q': // Quit
                run_program = 0;
                break;
        }
        key = 0;
    }

    // 4. Continuous Running Logic
    if (running && enabled) {
        unsigned int tick_delay = (unsigned int)(TIMER_FREQ / steps_per_sec);
        step_routine(direction, mode, motor_pins);
        step_timing_loop(tick_delay);
    } else {
        // Sleep briefly to prevent 100% CPU usage when idle
        usleep(10000);
    }
}

// 5. Cleanup before exit
// Drive the 4N25 optocoupler low and set to input so it floats[cite: 8]
enable_routine(PIN_ENABLE, 0);
E4235_Select(PIN_ENABLE, 0);[cite: 46, 56, 57]

// De-energize and float the motor coils
for (int i = 0; i < 4; i++) {
    E4235_Write(motor_pins[i], 0);[cite: 46, 57]
    E4235_Select(motor_pins[i], 0);[cite: 46, 56, 57]
}

E4235_KYBdeblock(0); // Restore blocking IO[cite: 46, 52, 53]
printf("\nExiting PARMCO safely.\n");
```

```
    return 0;
}
```

User prompt: could you also ensure to comment out any and all citations

Response: Here is the updated `main.c` file with all citations placed inside valid C comments so the code compiles correctly.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include "E4235.h" // Class library header[cite: 46]

// External assembly functions from motor_driver.s
extern void enable_routine(int pin, int state);
extern void step_routine(int direction, int mode, int* pins);
extern void step_timing_loop(unsigned int ticks);

// Define GPIO pins (Updated to match actual wiring)
#define PIN_ENABLE 24
#define PIN_A 17
#define PIN_B 27
#define PIN_C 22
#define PIN_D 23

// RP4 ARM Generic Timer frequency (typically 54 MHz)
#define TIMER_FREQ 54000000

// Global flag for the main loop, allowing the signal handler to gracefully exit
volatile int run_program = 1;

// Signal handler to catch Ctrl+C and ensure cleanup executes
void handle_sigint(int sig) {
    run_program = 0;
}

void test_motor_gpio(void) {
    int pins[4] = {17, 27, 22, 23};
    int driver_pins[4] = {2, 7, 10, 15};

    printf("\nStarting motor GPIO test...\n");

    // Keep the H-bridge disabled during testing
    E4235_Write(PIN_ENABLE, 0); //[cite: 46, 57]

    // Initialize all motor GPIOs LOW
    for (int i = 0; i < 4; i++) {
        E4235_Write(pins[i], 0); //[cite: 46, 57]
    }

    for (int i = 0; i < 4; i++) {

        // Set all motor GPIOs LOW
        for (int j = 0; j < 4; j++) {
            E4235_Write(pins[j], 0); //[cite: 46, 57]
        }

        // Set the GPIO being tested HIGH
        E4235_Write(pins[i], 1); //[cite: 46, 57]

        printf("\nGPIO%d is HIGH.\n", pins[i]);
        printf("Measure SN754410NE pin %d.\n", driver_pins[i]);
        printf("Expected voltage: approximately 3.3 V\n");
        printf("Press Enter when finished measuring...\n");

        getchar();
    }

    // Return all GPIOs LOW after testing
    for (int i = 0; i < 4; i++) {
        E4235_Write(pins[i], 0); //[cite: 46, 57]
    }
```

```
    printf("\nAll GPIO tests completed.\n");
}

int main() {
    // Register the signal handler for SIGINT (Ctrl+C)
    signal(SIGINT, handle_sigint);

    // 1. Initialize Hardware and GPIO
    if (E4235_Init() != 0) { //[cite: 46, 56]
        printf("Failed to initialize E4235 library.\n");
        return -1;
    }

    // Set motor and enable pins to OUTPUT (state = 1)
    E4235_Select(PIN_ENABLE, 1); //[cite: 46, 56, 57]
    int motor_pins[4] = {PIN_A, PIN_B, PIN_C, PIN_D};
    for (int i = 0; i < 4; i++) {
        E4235_Select(motor_pins[i], 1); //[cite: 46, 56, 57]
    }

    // 2. Initial State Variables
    int mode = 0;           // 0 = Full step, 1 = Half step
    int direction = 1;      // 1 = Clockwise, 0 = Counterclockwise
    int running = 0;        // 0 = Stopped, 1 = Continuous Run
    int enabled = 0;        // 0 = Disabled, 1 = Enabled
    float steps_per_sec = 100.0; // Initial step rate

    // Checkpoint 1 requirement: Print step mode at startup
    printf("PARMCO Motor Controller Initialized.\n");
    printf("Step Mode: %s\n", mode == 0 ? "Full Step (200/rev)" : "Half Step (400/rev)"); //[cite: 7]
    printf("Controls: [E]nable, [C]irection, [Space] One rev, [S]tart/Stop, [U] Faster, [D] Slower, [Q]uit\n")

    // Run the user-defined GPIO hardware test before unblocking the keyboard
    test_motor_gpio();

    // 3. Unblock standard input for non-blocking keyboard polling
    E4235_KYBdeblock(1); //[cite: 46, 52, 53]

    char key = 0;

    while (run_program) {
        // Read strictly 1 byte at a time in a WHILE loop to instantly drain the buffer
        while (read(STDIN_FILENO, &key, 1) > 0) {

            // Ignore trailing newlines or carriage returns
            if (key == '\n' || key == '\r') {
                continue;
            }

            switch (key) {
                case 'e':
                case 'E': // Enable/Disable H-Bridge
                    enabled = !enabled;
                    enable_routine(PIN_ENABLE, enabled);
                    printf("Motor %s.\n", enabled ? "ENABLED" : "DISABLED");
                    break;

                case 'c':
                case 'C': // Toggle Direction
                    direction = !direction;
                    printf("Direction: %s\n", direction ? "Clockwise" : "Counterclockwise");
                    break;

                case ' ': // Spacebar: One Revolution
                    if (enabled) {
                        printf("Executing one revolution...\n");
                        int steps_needed = (mode == 0) ? 200 : 400; //[cite: 7]
                        unsigned int tick_delay = (unsigned int)(TIMER_FREQ / steps_per_sec);
                        for (int i = 0; i < steps_needed; i++) {
                            step_routine(direction, mode, motor_pins);
                            step_timing_loop(tick_delay);
                        }
                        running = 0; // Ensure we stop after one revolution
```

```
                        printf("Revolution complete.\n");
                    } else {
                        printf("Cannot step: Motor is disabled.\n");
                    }
                    break;

                case 's':
                case 'S': // Continuous Run / Stop
                    running = !running;
                    printf("Motor %s.\n", running ? "RUNNING" : "STOPPED");
                    break;

                case 'u':
                case 'U': // Faster (increase by ~5%)
                    steps_per_sec *= 1.05; //[cite: 7]
                    printf("Speed increased. Current rate: %.2f steps/sec\n", steps_per_sec);
                    break;

                case 'd':
                case 'D': // Slower (decrease by ~5%)
                    steps_per_sec *= 0.95; //[cite: 7]
                    if (steps_per_sec < 10.0) steps_per_sec = 10.0; // Prevent stalling at zero
                    printf("Speed decreased. Current rate: %.2f steps/sec\n", steps_per_sec);
                    break;

                case 'q':
                case 'Q': // Quit
                    run_program = 0;
                    break;
            }
            key = 0;
        }

        // 4. Continuous Running Logic
        if (running && enabled) {
            unsigned int tick_delay = (unsigned int)(TIMER_FREQ / steps_per_sec);
            step_routine(direction, mode, motor_pins);
            step_timing_loop(tick_delay);
        } else {
            // Sleep briefly to prevent 100% CPU usage when idle
            usleep(10000);
        }
    }

    // 5. Cleanup before exit
    // Drive the 4N25 optocoupler low and set to input so it floats
    enable_routine(PIN_ENABLE, 0); //[cite: 8]
    E4235_Select(PIN_ENABLE, 0); //[cite: 46, 56, 57]

    // De-energize and float the motor coils
    for (int i = 0; i < 4; i++) {
        E4235_Write(motor_pins[i], 0); //[cite: 46, 57]
        E4235_Select(motor_pins[i], 0); //[cite: 46, 56, 57]
    }

    E4235_KYBdeblock(0); // Restore blocking IO[cite: 46, 52, 53]
    printf("\nExiting PARMCO safely.\n");
    return 0;
}
```

User prompt: still getting this bug

Response: This is a classic—and incredibly frustrating—embedded systems bug. Your terminal output shows the keystrokes being queued up (`S`, `S`, `S`, `E`, `E`) without triggering any prints from the program. This tells us the C program isn't blocking on `read()`; it has completely frozen inside the motor's continuous run logic and is never looping back to check the keyboard.

The culprit is hiding in your `step_timing_loop` in `motor_driver.s`.

The Root Cause: Signed Timer Wrap-Around

The Raspberry Pi 4's ARM generic timer (`CNTPCT`) runs at 54 MHz. We are reading the lower 32 bits of this timer.

In our original assembly code, we calculated a `target_time` and compared it to `current_time` using the `blt` (Branch if Less Than) instruction. The problem? **`blt` performs a *signed* comparison.**

Every 79 seconds, the lower 32 bits of a 54 MHz timer wrap past `0x7FFFFFFF` into what the processor considers "negative" numbers. When this happens, comparing a positive `target_time` to a negative `current_time` breaks the signed math, trapping the processor in an artificial 40-second infinite loop.

The Fix: Delta Timing with Unsigned Math

Instead of calculating a target time in the future, we need to calculate the *elapsed time* (`current - start`) and use `blo` (Branch if Lower) for an **unsigned** comparison. We also need to pad our `push`/`pop` instructions slightly to guarantee the stack remains strictly 8-byte aligned before branching to C functions, as required by the AAPCS standard.

Replace the code in your `motor_driver.s` file with this updated version:

```
.data
.align 2

@ Tracks the current position in the step sequence (0-7)
current_step:
    .word 0

@ Full-step coil patterns (4 steps: AB, BC, CD, DA)
@ Bits correspond to coils: 3=A, 2=B, 1=C, 0=D
full_step_table:
    .byte 0b1010, 0b0110, 0b0101, 0b1001

@ Half-step coil patterns (8 steps)
half_step_table:
    .byte 0b1000, 0b1010, 0b0010, 0b0110, 0b0100, 0b0101, 0b0001, 0b1001

.text
.global enable_routine
.global step_routine
.global step_timing_loop
.extern E4235_Write

@ ------------------------------------------------------------------------
@ enable_routine
@ Turns the H-bridge on or off through the 4N25 optocoupler.
@ ------------------------------------------------------------------------
enable_routine:
    @ Push an even number of registers (r4, lr) to maintain 8-byte AAPCS stack alignment[cite: 9]
    push {r4, lr}
    bl E4235_Write
    pop {r4, pc}

@ ------------------------------------------------------------------------
@ step_routine
@ Looks up the next coil pattern and drives the SN754410NE inputs.
@ ------------------------------------------------------------------------
step_routine:
    @ Push 6 registers (24 bytes) to maintain 8-byte AAPCS stack alignment[cite: 9]
    push {r4-r8, lr}

    mov r4, r0              @ Save direction
    mov r5, r1              @ Save mode
    mov r6, r2              @ Save pin array pointer

    @ Load current step index
    ldr r7, =current_step
    ldr r8, [r7]

    @ Update step index based on direction
    cmp r4, #1
    addeq r8, r8, #1        @ If CW, increment
    subne r8, r8, #1        @ If CCW, decrement

    @ Handle wrapping based on mode
    cmp r5, #1
    beq half_step_wrap

full_step_wrap:
    and r8, r8, #3          @ Wrap 0-3 for full step (modulo 4)
```

```
    ldr r4, =full_step_table
    b fetch_pattern

half_step_wrap:
    and r8, r8, #7         @ Wrap 0-7 for half step (modulo 8)
    ldr r4, =half_step_table

fetch_pattern:
    @ Store updated step index
    str r8, [r7]

    @ Fetch the current bit pattern from the table
    ldrb r4, [r4, r8]      @ r4 now holds the 4-bit pattern

    @ Drive Pin A (Bit 3)
    ldr r0, [r6, #0]
    and r1, r4, #0b1000
    lsr r1, r1, #3
    bl E4235_Write

    @ Drive Pin B (Bit 2)
    ldr r0, [r6, #4]
    and r1, r4, #0b0100
    lsr r1, r1, #2
    bl E4235_Write

    @ Drive Pin C (Bit 1)
    ldr r0, [r6, #8]
    and r1, r4, #0b0010
    lsr r1, r1, #1
    bl E4235_Write

    @ Drive Pin D (Bit 0)
    ldr r0, [r6, #12]
    and r1, r4, #0b0001
    bl E4235_Write

    pop {r4-r8, pc}

@ ---------------------------------------------------------------------------
@ step_timing_loop
@ Waits until the next step is due using the ARM generic timer[cite: 9].
@ ---------------------------------------------------------------------------
step_timing_loop:
    @ Push 4 registers (16 bytes) to maintain 8-byte AAPCS stack alignment[cite: 9]
    push {r4, r5, r6, lr}
    mov r4, r0             @ Save target delay in ticks

    @ Read the START time (CNTPCT lower half) into r5
    mrc p15, 0, r5, c14, c0, 0

wait_loop:
    @ Read the CURRENT time into r1
    mrc p15, 0, r1, c14, c0, 0

    @ Calculate elapsed time (Current - Start)
    sub r1, r1, r5

    @ Compare elapsed time against target delay
    cmp r1, r4

    @ blo (Branch if Lower) performs an UNSIGNED comparison,
    @ completely immune to the signed wrap-around bug!
    blo wait_loop

    pop {r4, r5, r6, pc}
```

Recompile using `make` and run the program again. This completely isolates the math from the 32-bit ceiling and should have your motor spinning smoothly without locking up your keystroke inputs.

User prompt: there is also no escape to the motor test function as well. can you please add that?

Response: Here is the updated `main.c` with an escape mechanism added to `test_motor_gpio()`. It now asks if you want to run the test at all, and allows you to type 'q' and press Enter to abort the test early while it is running.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include "E4235.h" // Class library header[cite: 46]

// External assembly functions from motor_driver.s
extern void enable_routine(int pin, int state);
extern void step_routine(int direction, int mode, int* pins);
extern void step_timing_loop(unsigned int ticks);

// Define GPIO pins (Updated to match actual wiring)
#define PIN_ENABLE 24
#define PIN_A 17
#define PIN_B 27
#define PIN_C 22
#define PIN_D 23

// RP4 ARM Generic Timer frequency (typically 54 MHz)
#define TIMER_FREQ 54000000

// Global flag for the main loop, allowing the signal handler to gracefully exit
volatile int run_program = 1;

// Signal handler to catch Ctrl+C and ensure cleanup executes
void handle_sigint(int sig) {
    run_program = 0;
}

void test_motor_gpio(void) {
    int pins[4] = {17, 27, 22, 23};
    int driver_pins[4] = {2, 7, 10, 15};
    char input[10];

    printf("\nDo you want to run the motor GPIO test? (y/n): ");
    if (fgets(input, sizeof(input), stdin) != NULL) {
        if (input[0] != 'y' && input[0] != 'Y') {
            printf("Skipping motor GPIO test.\n\n");
            return;
        }
    }

    printf("\nStarting motor GPIO test...\n");

    // Keep the H-bridge disabled during testing
    E4235_Write(PIN_ENABLE, 0); //[cite: 46, 57]

    // Initialize all motor GPIOs LOW
    for (int i = 0; i < 4; i++) {
        E4235_Write(pins[i], 0); //[cite: 46, 57]
    }

    for (int i = 0; i < 4; i++) {

        // Set all motor GPIOs LOW
        for (int j = 0; j < 4; j++) {
            E4235_Write(pins[j], 0); //[cite: 46, 57]
        }

        // Set the GPIO being tested HIGH
        E4235_Write(pins[i], 1); //[cite: 46, 57]

        printf("\nGPIO%d is HIGH.\n", pins[i]);
        printf("Measure SN754410NE pin %d.\n", driver_pins[i]);
        printf("Expected voltage: approximately 3.3 V\n");
        printf("Press Enter to continue, or 'q' then Enter to quit test...\n");

        if (fgets(input, sizeof(input), stdin) != NULL) {
            if (input[0] == 'q' || input[0] == 'Q') {
                printf("Aborting motor GPIO test.\n");
                break;
```

```
            }
        }
    }

    // Return all GPIOs LOW after testing
    for (int i = 0; i < 4; i++) {
        E4235_Write(pins[i], 0); //[cite: 46, 57]
    }

    printf("\nMotor GPIO test sequence finished.\n\n");
}

int main() {
    // Register the signal handler for SIGINT (Ctrl+C)
    signal(SIGINT, handle_sigint);

    // 1. Initialize Hardware and GPIO
    if (E4235_Init() != 0) { //[cite: 46, 56]
        printf("Failed to initialize E4235 library.\n");
        return -1;
    }

    // Set motor and enable pins to OUTPUT (state = 1)
    E4235_Select(PIN_ENABLE, 1); //[cite: 46, 56, 57]
    int motor_pins[4] = {PIN_A, PIN_B, PIN_C, PIN_D};
    for (int i = 0; i < 4; i++) {
        E4235_Select(motor_pins[i], 1); //[cite: 46, 56, 57]
    }

    // 2. Initial State Variables
    int mode = 0;           // 0 = Full step, 1 = Half step
    int direction = 1;      // 1 = Clockwise, 0 = Counterclockwise
    int running = 0;        // 0 = Stopped, 1 = Continuous Run
    int enabled = 0;        // 0 = Disabled, 1 = Enabled
    float steps_per_sec = 100.0; // Initial step rate

    // Checkpoint 1 requirement: Print step mode at startup
    printf("PARMCO Motor Controller Initialized.\n");
    printf("Step Mode: %s\n", mode == 0 ? "Full Step (200/rev)" : "Half Step (400/rev)"); //[cite: 7]
    printf("Controls: [E]nable, [C]irection, [Space] One rev, [S]tart/Stop, [U] Faster, [D] Slower, [Q]uit\n")

    // Run the user-defined GPIO hardware test before unblocking the keyboard
    test_motor_gpio();

    // 3. Unblock standard input for non-blocking keyboard polling
    E4235_KYBdeblock(1); //[cite: 46, 52, 53]

    char key = 0;

    while (run_program) {
        // Read strictly 1 byte at a time in a WHILE loop to instantly drain the buffer
        while (read(STDIN_FILENO, &key, 1) > 0) {

            // Ignore trailing newlines or carriage returns
            if (key == '\n' || key == '\r') {
                continue;
            }

            switch (key) {
                case 'e':
                case 'E': // Enable/Disable H-Bridge
                    enabled = !enabled;
                    enable_routine(PIN_ENABLE, enabled);
                    printf("Motor %s.\n", enabled ? "ENABLED" : "DISABLED");
                    break;

                case 'c':
                case 'C': // Toggle Direction
                    direction = !direction;
                    printf("Direction: %s\n", direction ? "Clockwise" : "Counterclockwise");
                    break;

                case ' ': // Spacebar: One Revolution
                    if (enabled) {
```

```
                        printf("Executing one revolution...\n");
                        int steps_needed = (mode == 0) ? 200 : 400; //[cite: 7]
                        unsigned int tick_delay = (unsigned int)(TIMER_FREQ / steps_per_sec);
                        for (int i = 0; i < steps_needed; i++) {
                            step_routine(direction, mode, motor_pins);
                            step_timing_loop(tick_delay);
                        }
                        running = 0; // Ensure we stop after one revolution
                        printf("Revolution complete.\n");
                    } else {
                        printf("Cannot step: Motor is disabled.\n");
                    }
                    break;

                case 's':
                case 'S': // Continuous Run / Stop
                    running = !running;
                    printf("Motor %s.\n", running ? "RUNNING" : "STOPPED");
                    break;

                case 'u':
                case 'U': // Faster (increase by ~5%)
                    steps_per_sec *= 1.05; //[cite: 7]
                    printf("Speed increased. Current rate: %.2f steps/sec\n", steps_per_sec);
                    break;

                case 'd':
                case 'D': // Slower (decrease by ~5%)
                    steps_per_sec *= 0.95; //[cite: 7]
                    if (steps_per_sec < 10.0) steps_per_sec = 10.0; // Prevent stalling at zero
                    printf("Speed decreased. Current rate: %.2f steps/sec\n", steps_per_sec);
                    break;

                case 'q':
                case 'Q': // Quit
                    run_program = 0;
                    break;
            }
            key = 0;
        }

        // 4. Continuous Running Logic
        if (running && enabled) {
            unsigned int tick_delay = (unsigned int)(TIMER_FREQ / steps_per_sec);
            step_routine(direction, mode, motor_pins);
            step_timing_loop(tick_delay);
        } else {
            // Sleep briefly to prevent 100% CPU usage when idle
            usleep(10000);
        }
    }

    // 5. Cleanup before exit
    // Drive the 4N25 optocoupler low and set to input so it floats
    enable_routine(PIN_ENABLE, 0); //[cite: 8]
    E4235_Select(PIN_ENABLE, 0); //[cite: 46, 56, 57]

    // De-energize and float the motor coils
    for (int i = 0; i < 4; i++) {
        E4235_Write(motor_pins[i], 0); //[cite: 46, 57]
        E4235_Select(motor_pins[i], 0); //[cite: 46, 56, 57]
    }

    E4235_KYBdeblock(0); // Restore blocking IO[cite: 46, 52, 53]
    printf("\nExiting PARMCO safely.\n");
    return 0;
}
```

User prompt: while the motor is running is when we encounter the keyboard input issue

Response: I see exactly what is happening based on your terminal screenshot. You pressed S to start the motor, the program printed "Motor RUNNING.", and then it completely froze. All the keystrokes you typed afterward (S, S, S, E, E) just piled up in the terminal because the C program

stopped polling for input.

The bug is not in the C keyboard code; it is a hardware-level infinite loop hiding in your `step_timing_loop` in `motor_driver.s`.

The Root Cause

In 32-bit ARM legacy OS mode, the instruction we initially used to read the timer (`mrc p15, 0, r1, c14, c0, 0`) actually reads the **Timer Frequency Register (`CNTFRQ`)**, not the current time!

Because the timer frequency is a constant hardware value (54,000,000), our start time and current time were always exactly the same. The elapsed time calculated by the subtraction was permanently stuck at `0`. Since `0` is always less than your target delay, the assembly loop immediately hung forever on the very first step, completely locking out your C program from ever checking the keyboard again.

To read the actual ticking clock—the 64-bit Physical Count Register (`CNTPCT`)—we must use the `mrrc` (Move to two Registers from Coprocessor) instruction instead.

The Fix

Replace the entire contents of your `motor_driver.s` file with this updated version. It correctly fetches the 64-bit timer using `mrrc`, extracts the lower 32 bits for the delta calculation, and uses unsigned `blo` branching to prevent wrap-around bugs.

```
.data
.align 2

@ Tracks the current position in the step sequence (0-7)
current_step:
    .word 0

@ Full-step coil patterns (4 steps: AB, BC, CD, DA)
@ Bits correspond to coils: 3=A, 2=B, 1=C, 0=D
full_step_table:
    .byte 0b1010, 0b0110, 0b0101, 0b1001

@ Half-step coil patterns (8 steps)
half_step_table:
    .byte 0b1000, 0b1010, 0b0010, 0b0110, 0b0100, 0b0101, 0b0001, 0b1001

.text
.global enable_routine
.global step_routine
.global step_timing_loop
.extern E4235_Write

@ ------------------------------------------------------------------------
@ enable_routine
@ Turns the H-bridge on or off through the 4N25 optocoupler.
@ ------------------------------------------------------------------------
enable_routine:
    @ Push an even number of registers (r4, lr) to maintain 8-byte AAPCS stack alignment[cite: 9]
    push {r4, lr}
    bl E4235_Write
    pop {r4, pc}

@ ------------------------------------------------------------------------
@ step_routine
@ Looks up the next coil pattern and drives the SN754410NE inputs.
@ ------------------------------------------------------------------------
step_routine:
    @ Push 6 registers (24 bytes) to maintain 8-byte AAPCS stack alignment[cite: 9]
    push {r4-r8, lr}

    mov r4, r0              @ Save direction
    mov r5, r1              @ Save mode
    mov r6, r2              @ Save pin array pointer

    @ Load current step index
    ldr r7, =current_step
    ldr r8, [r7]

    @ Update step index based on direction
    cmp r4, #1
    addeq r8, r8, #1
    subne r8, r8, #1

    @ Handle wrapping based on mode
```

```
    cmp r5, #1
    beq half_step_wrap

full_step_wrap:
    and r8, r8, #3
    ldr r4, =full_step_table
    b fetch_pattern

half_step_wrap:
    and r8, r8, #7
    ldr r4, =half_step_table

fetch_pattern:
    @ Store updated step index
    str r8, [r7]

    @ Fetch the current bit pattern from the table
    ldrb r4, [r4, r8]

    @ Drive Pin A (Bit 3)
    ldr r0, [r6, #0]
    and r1, r4, #0b1000
    lsr r1, r1, #3
    bl E4235_Write

    @ Drive Pin B (Bit 2)
    ldr r0, [r6, #4]
    and r1, r4, #0b0100
    lsr r1, r1, #2
    bl E4235_Write

    @ Drive Pin C (Bit 1)
    ldr r0, [r6, #8]
    and r1, r4, #0b0010
    lsr r1, r1, #1
    bl E4235_Write

    @ Drive Pin D (Bit 0)
    ldr r0, [r6, #12]
    and r1, r4, #0b0001
    bl E4235_Write

    pop {r4-r8, pc}

@ ------------------------------------------------------------------------
@ step_timing_loop
@ Waits until the next step is due using the ARM generic timer[cite: 8, 9].
@ ------------------------------------------------------------------------
step_timing_loop:
    @ Push 4 registers (16 bytes) to maintain 8-byte AAPCS stack alignment[cite: 9]
    push {r4, r5, r6, lr}
    mov r4, r0             @ Save target delay in ticks

    @ Read the 64-bit CNTPCT (Physical Count) into r5(lower) and r6(upper)
    mrrc p15, 0, r5, r6, c14

wait_loop:
    @ Read the 64-bit CNTPCT into r2(lower) and r3(upper)
    mrrc p15, 0, r2, r3, c14

    @ Calculate elapsed time (Current Lower - Start Lower)
    sub r2, r2, r5

    @ Compare elapsed time against target delay
    cmp r2, r4

    @ blo (Branch if Lower) performs an UNSIGNED comparison, immune to wraps
    blo wait_loop

    pop {r4, r5, r6, pc}
```

Recompile your object files with `make clean` and `make`. Once you run the program and hit `S`, the assembly will now properly wait 10 milliseconds and yield control back to C, bringing your keyboard inputs back to life.

User prompt: what is the pin out of the stepper motor?

Response: While the provided project documentation does not explicitly list the wire colors, the specified **Nema17 XY42STH34-0354A** stepper motor is a standard 4-wire bipolar stepper. Based on its datasheet, the typical pinout/wiring configuration is:

- **Coil 1 (Phase A):** Red and Yellow wire pair
- **Coil 2 (Phase B):** Green and Brown (or Gray) wire pair

Because it is a bipolar stepper, it requires both H-bridges on your SN754410NE chip. You will connect the Red/Yellow pair to the outputs of one H-bridge (e.g., outputs 1Y and 2Y) and the Green/Brown pair to the outputs of the second H-bridge (e.g., outputs 3Y and 4Y). If the motor spins in the opposite direction than expected during testing, you can simply swap the two wires of *one* of the coils (e.g., swap Red and Yellow) to reverse the polarity.

User prompt: when s is entered, what gpios output a signal?

Response: When you enter `s` (Start/Stop), the program toggles the continuous running state. Assuming the system has already been enabled (by pressing `e`), the C program begins repeatedly calling the `step_routine` assembly function in its main loop.

This function continuously cycles through the step sequence table and outputs alternating high and low signals to the four motor GPIO pins defined in your code:

- **GPIO 17** (`PIN_A`)
- **GPIO 27** (`PIN_B`)
- **GPIO 22** (`PIN_C`)
- **GPIO 23** (`PIN_D`)

These four GPIOs send the stepping sequence to the SN754410NE H-bridge to drive the motor coils.

GPIO 24 (`PIN_ENABLE`) also outputs a continuous high signal to keep the system active, but this pin is triggered when you press `e` to enable the H-bridge through the 4N25 optocoupler, not when you press `s`.

User prompt: when enable is toggled, the octocoupler outputs a 60hz sine wave to the enable pins of the hbridge

Response: If you are seeing a 60Hz sine wave on the oscilloscope, there is a hardware wiring issue. The enable signal must be strictly on/off (DC High/Low) and is explicitly prohibited from being PWM'd or analog.

A 60Hz sine wave indicates that the pin is picking up ambient AC mains noise because it is "floating" (acting as an antenna) rather than being actively driven to a known voltage.

- **Verify Common Ground:** Confirm there is solid continuity between the RP4 ground, the in-class DC supply ground, and the SN754410NE ground pins (4, 5, 12, and 13).
- **Check the 4N25 Output:** The optocoupler must actively drive the SN754410NE enable pins. If the optocoupler simply disconnects the circuit when turned off, the H-bridge enable pins will float and pick up room noise.
- **Add a Pull-Down Resistor:** You likely need a pull-down resistor on the H-bridge enable pins to tie them securely to ground when the 4N25 is not conducting. The project requires the hardware to guarantee the motor is disabled whenever the RP4 GPIO is low or floating.

User prompt: there is a capacitor connecting vcc1 and vcc2 of the hbridge to 5v and 12v

Response: If you have capacitors bridging directly *between* VCC1 and VCC2, you should remove them immediately.

While decoupling capacitors are excellent for stabilizing power, they must be wired from VCC1 to Ground and VCC2 to Ground individually. Bridging VCC1 and VCC2 together with a capacitor can couple high-voltage 12V motor transients directly into the 5V logic supply, which can cause erratic behavior or damage the SN754410NE.

Also, remember the strict hardware rule for Checkpoint 1: both the 5V logic supply (VCC1) and the 12V motor supply (VCC2) must come from the in-class DC power supply, never from the RP4.

While properly wired decoupling capacitors will help clean up high-frequency switching noise from the motor, they won't fix the 60Hz sine wave you are seeing on the enable pin. That 60Hz AC noise confirms the enable pin is floating. Adding a 10kΩ pull-down resistor from the H-bridge enable pins to ground will ensure they are actively pulled low whenever the 4N25 optocoupler turns off, fulfilling the requirement that the motor defaults to a disabled state.

User prompt: the ground was connected and now there is no sine wave. I am also seeing square wave inputs into all of the hbridge inputs. however no output is moving the motor

Response: That is excellent news. Seeing clean square waves on all four H-bridge inputs means your C controller, ARM assembly timing loop, and the Raspberry Pi GPIOs are working perfectly. The software side is solid.

Since the signals are reaching the SN754410NE but the motor isn't moving, the bottleneck is entirely on the power or output stage.

Check these specific hardware points:

- **Verify the Enable Voltage (Pins 1 and 9):** While the RP4 is outputting the enable signal, the 4N25 optocoupler must physically pass 5V to the SN754410NE enable pins to turn the outputs on. Press `E` in your program to enable the motor, and measure pins 1 and 9 on the H-bridge. If they are sitting at 0V instead of ~5V, your optocoupler circuit is not pulling the pins high.
- **Check the DC Supply Current Limit:** The Nema17 stepper draws about 0.35 A per phase. Check the in-class DC power supply screen. When you press `E` to enable the motor, the VCC2 current should clearly rise to roughly 0.5 A to 0.7 A. If the supply's current limit is set too low, it will choke the voltage the moment the coils try to draw power.
- **Probe VCC1 and VCC2 at the Chip:** Measure directly on the SN754410NE legs. Pin 16 (VCC1) must be 5.0 V and Pin 8 (VCC2) must be 12.0 V. If either is missing, the internal logic or the motor drive will fail to operate.
- **Probe the Output Pins (1Y, 2Y, 3Y, 4Y):** Check the actual output pins going to the motor. If you see 12V square waves here but the motor isn't moving, the motor coils are wired to the wrong pairs (e.g., mixing a Coil A wire with a Coil B wire). Coil 1 must be strictly on 1Y/2Y, and Coil 2 must be strictly on 3Y/4Y.

User prompt: i got the motor spinning however, there are a few bugs that need to get worked out

Response: Getting a stepper motor to spin for the first time is the biggest hurdle of the hardware phase, so excellent work crossing that line.

What specific bugs are you seeing now? Are you experiencing missed steps or erratic buzzing at higher RPMs, is the speed scaling behaving unexpectedly when you press 'U' or 'D', or are you running into issues executing the continuous run and stop commands?

User prompt: before we start, here is all of the updated code

Response: I have reviewed the updated `main.c`, `motor_driver.s`, and `Makefile`. The additions of the `T` test command for individual step progression and the `get_timer_frequency` assembly routine are integrated correctly to help isolate timing and logic issues.

Describe the specific bugs you are observing now that the motor is spinning. Are you experiencing skipped steps at higher RPMs, erratic behavior when changing directions, or issues with the `U` and `D` speed scaling commands?

User prompt: when the program is first started and before the motor is enabled, the motor turns freely like it is supposed to. after enabling the motor, it still turns freely, which it is not supposed to do. after starting the motor, it starts to spin, correct. after stopping, the motor does not spin freely, correct. after disabling the motor, the motor cannot move freely, incorrect. the motor wont turn freely until after the program is stopped.

Response: The symptoms you describe map perfectly to two separate issues: one in the software's initial state, and one in the hardware's TTL logic.

1. The Disable Bug (Hardware: Floating TTL Inputs) When you press `E` to disable the motor, your C program correctly sets the RP4 GPIO to `0`, turning off the 4N25 optocoupler. However, the SN754410NE is a TTL logic chip. When the 4N25 switch opens, the SN754410NE enable pins (1 and 9) are left disconnected, causing them to "float" to a logic HIGH state. Because they float HIGH, the H-bridge remains fully enabled and continues to drive the last known coil pattern to the motor, keeping it locked.

The only reason quitting the program (`Q`) frees the motor is because the cleanup routine forces the actual motor coils (pins A, B, C, and D) to 0V, dropping the holding torque.

- **The Hardware Fix:** To fulfill the project requirement that the motor is disabled whenever the optocoupler is off or floating, you must wire a pull-down resistor (e.g., 10kΩ) between the SN754410NE enable pins and ground. This actively pulls the voltage to 0V when the 4N25 turns off.

2. The Enable Bug (Software: Zeroed Coil State) When you first start the program, the `test_motor_gpio()` function leaves all four motor pins (A, B, C, D) at 0V. When you press `E` to enable the motor, the H-bridge turns on, but it applies 0V to all coils. A stepper motor only holds its position when a specific coil pattern (like `0b1010`) is applied. The motor turns freely because the coils are completely de-energized until you press `S`, which is the first time `step_routine` actually sends a holding pattern to the pins.

- **The Software Fix:** We need to update the `E` case in your `main.c` `switch` statement to immediately apply the current step pattern when enabled. We can also add a software failsafe to forcefully de-energize the coils when disabled, mimicking what the `Q` command does.

Update the `case 'e':` block inside your `main.c` `while` loop to match this:

```
                case 'e':
                case 'E': // Enable/Disable H-Bridge
                    enabled = !enabled;
                    enable_routine(PIN_ENABLE, enabled);

                    if (enabled) {
                        // Software Fix: Apply the current coil pattern immediately
                        // so the motor snaps into holding torque right when enabled.
```

```
            step_routine(direction, mode, motor_pins);
        } else {
            // Software Failsafe: De-energize all coils. This forces the
            // motor to free up even if the hardware enable pins are floating[cite: 8].
            for (int i = 0; i < 4; i++) {
                E4235_Write(motor_pins[i], 0); //[cite: 46, 57]
            }
        }

        printf("Motor %s.\n", enabled ? "ENABLED" : "DISABLED");
        break;
```

Compile and run this change. The motor will immediately lock when you press `E` initially, and the software failsafe will forcefully drop the holding torque when you disable it. Ensure your lab partner still adds the pull-down resistor to the circuit so the schematic passes the Checkpoint 1 hardware safety requirements.

User prompt: i think all functions are now working

Response: With the motor spinning and all keyboard commands functioning, the final hardware requirement to complete Checkpoint 1 is the stall-speed characterization.

To run this characterization using your C controller:

1. Ensure the propeller is securely fitted to the motor shaft.
2. Press `E` to enable and `S` to start the motor at a low initial step rate.
3. Press `U` repeatedly to increase the speed by 5% increments until the motor stalls, which will look like buzzing, erratic motion, or stopping entirely.
4. Note the final steps/second printed in your terminal just before the stall.
5. Convert that rate to RPM: `RPM = steps per second * 60 / steps per revolution` (200 for full-step, 400 for half-step).
6. Repeat this process for three separate runs to find your average stall speed, which must be compared against the 100 RPM limit.

Once your lab partner finishes collecting those numbers, would you like to begin drafting the human-written Checkpoint 1 Strategy Document or the Pre-Power Inspection PDF?

User prompt: what else is left for checkpoint 1

Response: For Checkpoint 1, your team still needs to complete and submit the required documentation and organize your repository.

Here are the remaining deliverables:

- **Strategy Document (Human-written PDF):** This document must detail your team members, phone platform, required resources, AI tools, AI-documentation plan, app plan, work division, and schedule.
- **Pre-Power Inspection (Human-written):** This must be documented as completed before power was first applied. It needs to verify that VCC1 and VCC2 come from the DC power supply rather than the RP4, check the common ground, SN754410NE pinout, 4N25 enable wiring, logic levels, and current/flyback protection, and record any problems found.
- **AI-Generated Hardware Documentation:** You must submit a preliminary motor schematic, assembly instructions, and a parts list. This package must be stored in the CP1 repository and submitted on ELC.
- **Submission and Repository Organization:** The human-written documents must be submitted on ELC as both a PDF and an editable version. Your CP1 repository must also be organized and clearly labeled.

Do you want to start outlining the Strategy Document, or should we prompt the AI to generate the preliminary schematic, assembly instructions, and parts list based on your lab partner's final wiring?

User prompt: can you make it display the step mode of the motor. like if it is running at full or half steops

Response: The Checkpoint 1 specifications require the step mode (full or half) to be printed at start-up, and the current step rate to be printed after every speed change. While the current code already prints the mode at initialization, you can append it to your continuous run and speed change outputs so it remains explicitly visible while the motor is operating.

Update the `S`, `U`, and `D` cases in your `main.c` `switch` statement to include the mode string:

```
        case 's':
        case 'S': // Continuous Run / Stop
            running = !running;
            printf("Motor %s in %s mode.\n", running ? "RUNNING" : "STOPPED", mode == 0 ? "Full Step"
```

```
            break;

        case 'u':
        case 'U': // Faster (increase by ~5%)
            steps_per_sec *= 1.05; //[cite: 7]
            printf("Speed increased. Current rate: %.2f steps/sec (%s)\n", steps_per_sec, mode == 0 ?
            break;

        case 'd':
        case 'D': // Slower (decrease by ~5%)
            steps_per_sec *= 0.95; //[cite: 7]
            if (steps_per_sec < 10.0) steps_per_sec = 10.0; // Prevent stalling at zero
            printf("Speed decreased. Current rate: %.2f steps/sec (%s)\n", steps_per_sec, mode == 0 ?
            break;
```

User prompt: when using step and t to advance to the next step, the motor is disabled and does not move

Response: The motor is not moving because the code for the T command explicitly disables the H-bridge before taking a step. In your current `main.c`, the T case sets `enabled = 0;` and calls `enable_routine(PIN_ENABLE, 0);`.

While this is useful for safely measuring the logic signals on the H-bridge inputs with a multimeter without drawing current, it prevents the SN754410NE from energizing the motor coils to physically move the shaft.

To make the motor physically advance one step when you press T, the H-bridge must remain enabled. Update the T case in your `switch` statement to ensure the motor is enabled before stepping:

```
        case 't':
        case 'T': // Test one assembly motor step
            running = 0; // Stop continuous run

            // To physically move the motor, the H-bridge must be enabled
            if (!enabled) {
                enabled = 1;
                enable_routine(PIN_ENABLE, 1);
                printf("Motor automatically ENABLED for step.\n");
            }

            // Advance the assembly stepping sequence once
            step_routine(direction, mode, motor_pins);

            printf("\nAssembly step executed.\n");
            printf("Direction: %s\n", direction ? "Clockwise" : "Counterclockwise");
            printf("Motor stepped. Measure SN754410NE input pins 2, 7, 10, and 15 if needed.\n");
            printf("Press T to advance to the next step.\n\n");
            break;
```

User prompt: could you make it so that it does not affect the motor enable or disable control? t will only move the motor when the motor is already enabled. if the motor is disabled, it will not move the motor

Response: Here is the updated T case. It now mirrors the logic of your spacebar command by simply checking the `enabled` flag first. If the motor is disabled, it will halt the continuous run but refuse to step the motor, leaving your enable/disable state completely untouched.

Replace the `case 't':` block in your `switch` statement with this:

```
        case 't':
        case 'T': // Test one assembly motor step
            running = 0; // Stop continuous run

            if (enabled) {
                // Advance the assembly stepping sequence once
                step_routine(direction, mode, motor_pins);

                printf("\nAssembly step executed.\n");
                printf("Direction: %s\n", direction ? "Clockwise" : "Counterclockwise");
                printf("Motor stepped. Measure SN754410NE input pins 2, 7, 10, and 15 if needed.\n");
                printf("Press T to advance to the next step.\n\n");
            } else {
                printf("\nCannot step: Motor is currently DISABLED and will not move.\n");
                printf("Press E to enable the motor first.\n\n");
            }
            break;
```

User prompt: i changed the code to use half steps for testing purposed. the output of xxx steps per second for a half step, is that half steps per second? or full steps?

Response: It refers to **half steps per second**.

The `steps_per_sec` variable dictates exactly how many times your C program calls the assembly `step_routine` each second. Because you switched to half-step mode, every single call to that routine now advances the sequence by just one half-step (cycling through the 8-step table instead of the 4-step table).

If your terminal prints "100 steps/sec" while in half-step mode, the motor is executing 100 half-steps every second. Since a standard Nema17 requires 400 half-steps to complete a single 360-degree revolution, running at 100 steps/sec in this mode means it will take exactly 4 seconds to turn one full rotation (15 RPM).

User prompt: we need to make some tweaks to the code. currently, we are only acheiving about 100 rpm. is there a way to bring up the speed of the motor before it begins to stall

Response: Change the initial `mode` variable in `main.c` back to `0` (Full Step). Half-stepping provides smoother motion at low speeds but reduces overall torque because only one coil is energized during the intermediate steps. Full-stepping keeps both coils fully energized at all times, providing the maximum possible magnetic holding force to push through higher RPMs before stalling.

Ensure you are ramping up the speed gradually. Stepper motors cannot instantly jump to a high speed from a dead stop due to rotor inertia. Start the motor at a low speed (e.g., 50 steps/sec) and wait for it to stabilize. Then, press `U` slowly, allowing the motor to fully accelerate to the new speed before pressing `U` again. If you increase the step rate faster than the rotor can physically accelerate, it will instantly stall and buzz.

Modify the `U` command to use a smaller multiplier for a gentler acceleration curve at high speeds. Changing the 5% increase to a 1% or 2% increase allows you to inch closer to the physical limit without overshooting it:

```
            case 'u':
            case 'U':
                steps_per_sec *= 1.02; // Change to 2% increase for finer high-speed control
                printf("Speed increased. Current rate: %.2f steps/sec (%s)\n", steps_per_sec, mode == 0 ?
                break;
```

Be aware that ~100 to 150 RPM is often the physical ceiling for a Nema 17 motor driven by a basic SN754410NE TTL H-bridge at 12V. Modern 3D printers achieve higher speeds using specialized "chopper" drivers (like the A4988) running at 24V, which aggressively force high voltage into the coils to overcome electromagnetic inductance. Because your project is restricted to a 12V DC supply and a simple H-bridge, the coil inductance physically prevents the current from rising fast enough during very short step pulses, starving the motor of torque at higher speeds.

User prompt: currently one full rotation is off by about 5 degrees in either direction

Response: A 5-degree error means your motor is missing or overshooting exactly 2 to 3 steps per revolution (since a Nema17 motor moves 1.8 degrees per full step).

This is almost certainly caused by **rotor inertia**, not a math error in your 200-step loop.

When you press the spacebar, your `for` loop commands the motor to instantaneously jump from a dead stop to your current `steps_per_sec` rate. The magnetic field in the stator begins rotating at full speed immediately, but the physical mass of the rotor (and the attached propeller) takes a fraction of a second to overcome inertia. The rotor lags behind and physically skips the first 2 or 3 magnetic steps before it catches up to the rotating field. It can also overshoot by a step or two when the loop abruptly ends.

Here is how to fix and verify this:

1. The Mechanical Check First, ensure the set screw on the propeller hub is tightened against the flat side of the motor's D-shaft. If it is fastened against the round part of the shaft, the sudden jerk of starting and stopping will cause the propeller to slip exactly a few degrees on the metal shaft.

2. The Software Fix (Inertia Profiling) To prove this is an inertia problem, press `D` repeatedly to bring your speed down to roughly 30 steps/sec, then press the spacebar. At this low speed, the motor has enough torque to overcome inertia instantly without skipping steps. It should hit 360 degrees perfectly.

To permanently fix the one-revolution command so it is consistently accurate regardless of your current running speed, you can decouple it from the `steps_per_sec` variable and assign it a safe, fixed speed. Alternatively, you can add a simple acceleration ramp to the spacebar logic.

Here is an updated `case ' ':` block that adds a basic acceleration/deceleration curve (ramp-up and ramp-down) to the single revolution to prevent missed steps:

```
            case ' ': // Spacebar: One Revolution
                if (enabled) {
                    printf("Executing one revolution with acceleration profile...\n");
                    int steps_needed = (mode == 0) ? 200 : 400;

                    // Start slow, peak at a target speed, then slow down
                    float current_rev_speed = 20.0; // Start at 20 steps/sec
                    float target_rev_speed = 100.0; // Peak at 100 steps/sec
```

```
            for (int i = 0; i < steps_needed; i++) {
                // Acceleration Phase (first 25% of steps)
                if (i < steps_needed / 4 && current_rev_speed < target_rev_speed) {
                    current_rev_speed += 2.0;
                }
                // Deceleration Phase (last 25% of steps)
                else if (i > (steps_needed * 3) / 4 && current_rev_speed > 20.0) {
                    current_rev_speed -= 2.0;
                }

                unsigned int tick_delay = (unsigned int)(TIMER_FREQ / current_rev_speed);

                step_routine(direction, mode, motor_pins);
                step_timing_loop(tick_delay);
            }
            running = 0; // Ensure we stop after one revolution
            printf("Revolution complete.\n");
        } else {
            printf("Cannot step: Motor is disabled.\n");
        }
        break;
```

User prompt: this happens when testing using the single rotation option.

Response: If the 5-degree error specifically happens during the single rotation (spacebar) command, that perfectly confirms the issue is rotor inertia.

When you trigger that single rotation, the code instantaneously attempts to drive the motor at full speed from a dead stop, and then instantly stops it 200 steps later. The magnetic field rotates exactly 360 degrees, but the physical mass of the rotor and propeller skips a couple of steps at the very beginning (because it can't accelerate fast enough) and likely overshoots at the very end (because of momentum).

How to verify and fix this right now:

1. **Apply the Acceleration Code:** If you haven't already, paste the updated `case ' ':` block from my previous message into your `main.c`. This adds a ramp-up and ramp-down curve so the motor starts gently, hits its peak speed, and slows down before stopping.
2. **Test at a Crawl:** Press `D` until your speed is around 20 or 30 steps/sec, then press the spacebar. At this slow speed, the motor has maximum torque and near-zero inertial lag. If it completes a perfect 360 degrees here, you know for sure the motor is skipping steps at higher speeds.
3. **Tighten the Propeller Set Screw:** If the error is *exactly* 5 degrees every single time, regardless of speed, and happens in both directions, you have mechanical backlash. Check the set screw on the propeller hub. It must be tightened flat against the "D" shaped cut-out on the motor shaft, not on the rounded edge. If it is on the round edge, the shaft will slip a few degrees inside the hub every time it starts or changes direction.

Let me know what happens when you run the spacebar command at a very low speed!

User prompt: now its undershooting by about 5 degrees

Response: An undershoot of 5 degrees means the motor is falling short by roughly 3 full steps (1.8 degrees per step). Because your `for` loop mathematically guarantees exactly 200 steps are sent to the H-bridge, the magnetic field inside the motor is rotating a perfect 360 degrees. The physical rotor is simply falling 3 steps behind the magnetic field and never catching up.

This points to one of two things: the acceleration profile is still slightly too steep for the propeller's mass, or you have mechanical slippage.

To isolate the root cause, run a **Constant Crawl Test**. Replace the spacebar block with this ultra-slow, constant-speed version that eliminates all inertia variables:

```
        case ' ': // Spacebar: One Revolution (Crawl Test)
            if (enabled) {
                printf("Executing slow constant-speed revolution to test mechanics...\n");
                int steps_needed = (mode == 0) ? 200 : 400;

                // Force a very slow 20 steps/sec speed with NO acceleration ramp
                unsigned int tick_delay = (unsigned int)(TIMER_FREQ / 20.0);

                for (int i = 0; i < steps_needed; i++) {
                    step_routine(direction, mode, motor_pins);
                    step_timing_loop(tick_delay);
                }
                running = 0;
                printf("Revolution complete.\n");
            } else {
                printf("Cannot step: Motor is disabled.\n");
```

```
                }
                break;
```

- **If the crawl test perfectly hits 360 degrees:** The software is mathematically sound, but the previous acceleration ramp was still too aggressive for the physical inertia. Revert to the acceleration code and flatten the curve (e.g., change `current_rev_speed += 2.0` to `+= 1.0`, or lower the `target_rev_speed`).
- **If the crawl test ALSO undershoots:** The issue is entirely mechanical. The motor shaft is turning 360 degrees, but the propeller is slipping. Double-check that the set screw on the propeller hub is tightened flush against the flat "D" cut-out of the motor shaft. If it is fastened against the rounded edge, the torque of starting and stopping forces it to slide exactly a few degrees every time.

User prompt: still undershooting it. make it go about 10% fasTER

Response:

```
            case ' ': // Spacebar: One Revolution (10% Faster)
                if (enabled) {
                    printf("Executing single revolution at 22 steps/sec...\n");
                    int steps_needed = (mode == 0) ? 200 : 400;

                    // Increased from 20.0 to 22.0 steps/sec (10% faster)
                    unsigned int tick_delay = (unsigned int)(TIMER_FREQ / 22.0);

                    for (int i = 0; i < steps_needed; i++) {
                        step_routine(direction, mode, motor_pins);
                        step_timing_loop(tick_delay);
                    }
                    running = 0;
                    printf("Revolution complete.\n");
                } else {
                    printf("Cannot step: Motor is disabled.\n");
                }
                break;
```

If the motor still physically undershoots after this change, you have conclusively proven the issue is a mechanical hardware fault, not a software bug.

Your C `for` loop strictly executes exactly 200 times. A standard Nema17 stepper motor moves exactly 1.8 degrees per full step. Mathematically, 200 steps × 1.8 degrees = 360.0 degrees of magnetic rotation.

Because the software is sending a perfect 360 degrees of rotation, but the propeller is stopping at ~355 degrees, the propeller is physically slipping on the motor shaft. Verify that the set screw on the propeller hub is tightened flush against the flat "D" cut-out of the motor shaft. If the screw is clamped against the round edge of the shaft, the torque of starting and stopping forces the propeller to slide exactly a few degrees on the metal before catching.

User prompt: counter clockwise is a little short

Response: If the rotation is falling short specifically when you switch to counter-clockwise, you are experiencing mechanical **backlash** (physical slack).

Because your assembly math handles negative wrap-around flawlessly (`0xFFFFFFFF AND 3 = 3`), the magnetic sequence inside the motor is perfect.

Here is what is physically happening: When you reverse direction, the metal motor shaft begins turning immediately. However, because the set screw on the propeller is slightly loose, the shaft spins freely for about 5 degrees *inside* the propeller hub before it finally catches against the screw and begins pushing the propeller. The motor shaft completed a perfect 360 degrees, but the propeller only completed 355.

The "Twice in a Row" Test to prove it:

1. Set the direction to Counter-Clockwise (`C`).
2. Press the spacebar. Note that it falls a little short.
3. **Without changing direction**, press the spacebar again.

If that second Counter-Clockwise rotation is a perfect 360 degrees, it is 100% mechanical backlash. The first rotation "ate up" the slack, so the second rotation had a solid grip the entire time.

The Fix: You must grab the allen wrench and tighten the set screw on the propeller hub. Ensure the screw is pointing directly at the **flat "D" cut-out** of the motor shaft, not the rounded edge. If it is on the rounded edge, the propeller will always rock back and forth by a few degrees every time you reverse direction.

User prompt: moving onto checkpoint 2, I am in charge of wiring up the IR sensor and calculating the motor's rpms